

RAMAIAH INSTITUTE OF TECHNOLOGY, BANGALORE – 560054

(Autonomous Institute, Affiliated to VTU)

Department of Computer Science & Engineering

Practical Assignment Report

FacultyDesk: A Scheduling, Reminder and Task Management System for College Faculty

CSE556: Object-Oriented Analysis and Design

SUBMITTED BY:

Name	USN	Sec
Krish Kumar Sharma	1MS25CS408	5 'C'
Thanishq Vijaykumar	1MS24CS202	5 'D'
Santhosha Naik D K	1MS25CS416	5 'D'

SUPERVISED BY:

Faculty

DR. PARKAVI A

PROFESSOR

DEPARTMENT OF CSE

Sep – Nov, 2026

Department of Computer Science and Engineering

M S Ramaiah Institute of Technology

(Autonomous Institute, Affiliated to VTU)

Bangalore – 54

CERTIFICATE

This is to certify that **Krish Kumar Sharma (1MS25CS408)**, **Thanishq Vijaykumar (1MS24CS202)** and **Santhosha Naik D K (1MS25CS416)** have completed the "**FacultyDesk: A Scheduling, Reminder and Task Management System for College Faculty**" as part of the Practical Assignment for the course **Object-Oriented Analysis and Design (CSE556)**. We declare that the entire content embodied in this B.E. CSE, 5th semester report is not copied.

Submitted by

Krish Kumar Sharma (1MS25CS408)

Thanishq Vijaykumar (1MS24CS202)

Santhosha Naik D K (1MS25CS416)

Guided by

Dr. Parkavi A

Professor, Dept. of CSE

Department of Computer Science and Engineering

M S Ramaiah Institute of Technology

(Autonomous Institute, Affiliated to VTU)

Bangalore – 54

Evaluation Sheet

USN	Name	OOAD approach & Explanation Skills (5)	Coding & Documentation (5)	Total Marks (10)
1MS25CS408	Krish Kumar Sharma			
1MS24CS202	Thanishq Vijaykumar			
1MS25CS416	Santhosha Naik D K			

Evaluated By

Name: Dr. Parkavi A

Designation: Professor

Department: Computer Science & Engineering (AIML) & CSE (CS), RIT

Signature:

Table of Contents

1 Abstract **9**

2 Introduction **10**

2.1 Background 10

2.2 Existing System and its Limitations 10

2.3 Proposed System 10

2.4 Technology Used 11

2.5 Organisation of the Report 11

3 Objectives & Problem Statement **12**

3.1 Problem Statement 12

3.2 Objectives 12

3.3 Functional Requirements 12

3.4 Non-Functional Requirements 13

3.5 Scope and Constraints 14

4 Object Basics and Project Overview **15**

4.1 Object Basics 15

4.2 System Overview 16

4.3 Project Modules 16

4.4 Business Aspects 18

5 UML Modelling **20**

5.1 Use Case Diagram 20

5.2 Class Diagram 26

5.3 Sequence Diagrams 30

5.4 Collaboration (Communication) Diagram 33

5.5 Statechart Diagrams 34

5.6 Activity Diagram 36

5.7 Package Diagram 38

5.8 Component Diagram 41

5.9 Deployment Diagram 42

5.10 Entity–Relationship Diagram 44

6 Object-Oriented Analysis, Design & Architecture **45**

6.1 Object-Oriented Analysis 45

6.2 Object-Oriented Design 46
6.3 Architecture Decisions 49

7 Implementation (Algorithms Pseudo Code) 50
7.1 Implementation Overview 50
7.2 Algorithm 1: Three-Tier Initials Resolution 50
7.3 Algorithm 2: Sweep-Line Clash Detection 51
7.4 Algorithm 3: Weekly Class Generation 52
7.5 Algorithm 4: Reminder Planning with Quiet Hours 52
7.6 Algorithm 5: Reminder Dispatch 52
7.7 Algorithm 6: Task Priority Score 53
7.8 Algorithm 7: Circular Review Gate 53

8 Results 55
8.1 Automated Backend Tests 55
8.2 End-to-End Browser Tests 55
8.3 Algorithm Outputs on Worked Examples 56
8.4 Clash Detection Performance 57
8.5 Database Benchmark (Recorded Earlier) 57
8.6 Application Screens 57

9 Discussion of Results 61
9.1 What the Results Show 61
9.2 Clash Detection 61
9.3 Initials Matching 61
9.4 Limitations Found During Testing 61
9.5 Comparison with the 2026 Literature 62

10 Conclusion 63
10.1 Future Work 63

11 References 64

12 Literature Survey Proofs 65
12.1 Automated University Timetabling: An AI Approach for Efficient Schedule Management 65
12.2 Course Timetabling under Conflict and Lecturer Preferences using Bi-objective Optimization Model 66

12.3 OCR or Not? Rethinking Document Information Extraction in the MLLMs Era with Real-World Large-Scale Datasets 67
12.4 A Multistage Extraction Pipeline for Long Scanned Financial Documents: An Empirical Study in Industrial KYC Workflows 68
12.5 Operationalizing Document AI: A Microservice Architecture for OCR and LLM Pipelines in Production 68

List of Figures

1 High-level architecture of FacultyDesk 16
2 FacultyDesk use case diagram 22
3 FacultyDesk domain class diagram 29
4 Sequence diagram: timetable upload and class generation 31
5 Sequence diagram: circular upload, OCR, review and confirmation 31
6 Sequence diagram: periodic reminder dispatch 32
7 Collaboration diagram: HOD confirms a meeting circular 33
8 Statechart diagram of Circular 35
9 Statechart diagram of ScheduledNotification 37
10 Statechart diagram of TimetableUpload 37
11 Activity diagram: timetable ingestion and initials resolution 39
12 Package diagram with import dependencies 40
13 Component diagram with provided interfaces 41
14 Deployment diagram (production on a department server) 43
15 Entity–relationship diagram of the PostgreSQL schema 44
16 Layered architecture of the backend 47
17 Class diagram of the Strategy, Factory and Composite patterns 48
18 Class diagram of the Repository and Unit of Work patterns and the injected Clock 49
19 Clash detection time against the number of events 58
20 Faculty "Today" screen: next meeting countdown, today's schedule and high-priority tasks 58
21 Documentation tasks ranked by priority score P (overdue tasks have $P > 1$) . . 59
22 HOD timetable upload screen 59
23 HOD circular review queue with OCR-extracted date, time and venue 60
24 Reminder settings: web push, quiet hours and private ICS feed 60
25 Proof: official page of "Automated University Timetabling: An AI Approach for Efficient Schedule Management" 65
26 Proof: official page of "Course Timetabling under Conflict and Lecturer Preferences using Bi-objective Optimization Model" 66
27 Proof: official page of "OCR or Not? Rethinking Document Information Extraction in the MLLMs Era with Real-World Large-Scale Datasets" 67
28 Proof: official page of "A Multistage Extraction Pipeline for Long Scanned Financial Documents: An Empirical Study in Industrial KYC Workflows" 68
29 Proof: official page of "Operationalizing Document AI: A Microservice Architecture for OCR and LLM Pipelines in Production" 69

List of Tables

1 Technology stack . . . 11
2 Functional requirements . . . 12
3 Non-functional requirements . . . 14
4 Object basics with project examples . . . 15
5 Stakeholders and benefits . . . 18
6 Business process comparison . . . 18
7 Risks and controls . . . 19
8 UML diagrams presented in this report . . . 20
9 Use case diagram notation . . . 21
10 Actors and use cases . . . 23
11 Use case scenario: Upload and Confirm Timetable . . . 24
12 Use case scenario: Upload and Confirm Meeting Circular . . . 25
13 Use case scenario: Send Reminders . . . 25
14 Use case scenario: Assign and Complete Documentation Task . . . 26
15 Class relationship notation . . . 27
16 Multiplicity notation . . . 27
17 Domain classes with attributes and operations . . . 27
18 Sequence diagram notation . . . 30
19 Sequence versus collaboration diagram . . . 33
20 Messages in the collaboration diagram . . . 34
21 Statechart notation . . . 35
22 State transition table: Circular . . . 35
23 State transition table: ScheduledNotification . . . 36
24 State transition table: TimetableUpload . . . 36
25 Activity diagram notation . . . 37
26 Activities in timetable ingestion . . . 37
27 Components and interfaces . . . 41
28 Classification of candidate objects . . . 45
29 Filtering of candidates . . . 45
30 Design principles . . . 47
31 Design patterns used . . . 48
32 Architecture decisions . . . 49
33 Modules and source files . . . 50
34 Backend test results by file . . . 55
35 End-to-end test results . . . 56
36 Priority scores of four sample tasks . . . 56
37 Normalised Levenshtein similarity examples . . . 56
38 Sweep-line versus brute-force clash detection (one run per size) . . . 57
39 Limitations and planned fixes . . . 62

1. Abstract

Faculty members in an engineering department follow three separate sources of information: the department timetable for classes, paper circulars for meetings, and verbal or e-mail instructions for documentation work such as CO–PO mapping, internal assessment (IA) marks upload and lab manual updates. None of these sources reminds anyone, clashes between duties are found only when they happen, and the Head of the Department (HOD) has no single view of pending documentation.

FacultyDesk is a web application that brings all three sources into one personal calendar for every faculty member. The HOD uploads the timetable file the department already produces (Excel, CSV or PDF); the system reads every cell, matches teacher initials such as `CN (KKS)` to faculty accounts and generates each person's weekly classes for the term, skipping holidays. Any faculty member can photograph a meeting circular; optical character recognition (OCR) extracts the date, time, venue and the faculty named in it, and uncertain results go to a review queue where the HOD confirms them. Confirmed meetings appear on every attendee's calendar with clash detection. Reminders are sent by e-mail and web push at times each user chooses, respect quiet hours, and are sent at most once per reminder. A task tracker ranks documentation work by a priority score, alerts before each deadline and escalates overdue work to the HOD.

The system was designed using Object-Oriented Analysis and Design (OOAD). Twenty domain classes carry the business rules; three of them (`Circular`, `ScheduledNotification` and `TimetableUpload`) are explicit state machines. The code follows a four-layer architecture (domain, application, infrastructure, API) and applies the Repository, Unit of Work, Strategy, Factory, Composite and State design patterns. This report presents the business case, the object-oriented analysis, eleven kinds of UML and related diagrams with explanations and use case scenarios, the core algorithms in pseudo code, and the results.

On 13 November 2026, all 83 automated backend tests and all 4 browser end-to-end tests passed on the running system. The sweep-line clash detector gave exactly the same results as a brute-force check on up to 4,000 events and was 121 times faster at that size.

Keywords: Object-oriented analysis and design, UML, use case, class diagram, design patterns, clean architecture, OCR, scheduling, reminders, academic administration.

2. Introduction

2.1 Background

A typical working week of a faculty member combines three kinds of commitments. **Classes** come from the department timetable, a spreadsheet grid in which each cell holds a course code, a section and the teacher's initials, for example `CN (KKS)`. **Meetings** are announced through paper circulars that are signed, scanned and pinned on the notice board or forwarded as a photo in a group chat. **Documentation work**, such as CO–PO (course outcome to programme outcome) mapping, IA marks upload, attendance shortage reports and lab manual updates, arrives as instructions with a deadline.

2.2 Existing System and its Limitations

In the current manual process each faculty member keeps track of these items personally.

- **No single view.** Classes, meetings and deadlines live in three different places.
- **No reminders.** The timetable, the notice board and the e-mail do not remind anyone.
- **Clashes are found late.** A meeting called during a scheduled class is noticed only on the day.
- **Revisions are missed.** When the timetable changes, every teacher has to read the new sheet again.
- **No follow-up for documentation.** The HOD has to ask each person whether a task is done.
- **Retyping.** General calendar apps require every class and meeting to be entered by hand.

2.3 Proposed System

FacultyDesk reads the documents the department already produces instead of asking anyone to retype them.

1. The HOD uploads the timetable file. The system parses it, matches initials to faculty accounts, shows a preview, and after confirmation generates every class of the term for every teacher.
2. Anyone uploads a photo or scan of a circular. OCR reads it and proposes a meeting with date, time, venue and attendees. The HOD reviews uncertain results and confirms.
3. Every faculty member sees classes, meetings and deadlines on one calendar with clash warnings, and can subscribe to it from Google Calendar or Apple Calendar through an iCalendar (ICS) feed [8].

4. Reminders arrive on the phone (web push) and by e-mail at chosen lead times, outside the user's quiet hours.
5. Documentation tasks are tracked with deadlines, a priority score, alerts and escalation to the HOD.

2.4 Technology Used

Table 1: Technology stack

Layer	**Technology**
Backend	Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2 (async), Alembic migrations
Background jobs	Celery 5 with Redis 7 (worker and Celery Beat scheduler)
Database and files	PostgreSQL 16; MinIO (S3-compatible object storage)
OCR and parsing	Tesseract OCR [6], OpenCV, pdfplumber, openpyxl, rapidfuzz
Notifications	SMTP e-mail (aiosmtplib), Web Push with VAPID keys (pywebpush)
Frontend	Next.js (App Router), TypeScript, Tailwind CSS, FullCalendar, installable PWA
Security	Argon2id password hashing, JWT access tokens, rotating refresh cookies
Quality	ruff, mypy (strict), pytest, Playwright end-to-end tests
Deployment	Docker Compose, Caddy reverse proxy with HTTPS

2.5 Organisation of the Report

Section 3 states the problem, objectives and requirements. Section 4 explains the object-oriented basics used, the eight modules of the project and the business aspects. Section 5 presents the UML models, each with theory, diagram, explanation and tables. Section 6 covers the object-oriented analysis, design and architecture. Section 7 gives the core algorithms in pseudo code. Sections 8 and 9 present and discuss the results, and Section 10 concludes. Section 12 lists the 2026 papers surveyed, with proof.

3. Objectives & Problem Statement

3.1 Problem Statement

A college department wants a system that turns its existing timetable files and paper meeting circulars into a personal, clash-aware calendar for every faculty member. The HOD uploads the department timetable; the system must identify each teacher from the initials written in the timetable cells and create that teacher's classes for the whole term, leaving out holidays. Any faculty member may upload a photo of a meeting circular; the system must read the date, time, venue and the names or initials of the invited faculty, and must not place a meeting on anyone's calendar until it is sure of the details or the HOD has confirmed them. Every faculty member must be reminded before each class, meeting and deadline by e-mail or phone notification, at times they choose, but not during their quiet hours, and never twice for the same reminder. The HOD must be able to assign documentation tasks to one teacher, a group or the whole department, see who has completed them, and be alerted when work is overdue.

3.2 Objectives

1. **Timetable ingestion:** read Excel, CSV and text-layer PDF timetables, expand merged period cells, resolve initials to faculty accounts, and generate the term's weekly classes while skipping declared holidays.
2. **Circular scanning:** accept photos, scans and PDFs of circulars, extract date, time, venue, agenda and attendees with OCR, and send uncertain extractions to an HOD review queue.
3. **Personal calendar:** show classes, meetings and deadlines in one view, detect overlapping events, find common free slots for a group, and publish a private ICS feed.
4. **Reminders:** deliver reminders by e-mail and web push at user-chosen lead times, honour quiet hours, retry failed sends and never send the same reminder twice.
5. **Documentation tasks:** create and assign tasks, rank them by a transparent priority score, alert before the deadline and escalate overdue work.
6. **Object-oriented design:** model the domain with classes that hold their own rules, keep business logic independent of frameworks, and document the design in UML.

3.3 Functional Requirements

Table 2: Functional requirements

ID	Requirement	Actor

FR-01	Log in with e-mail and password; access depends on role (Admin, HOD, Faculty).	All
FR-02	Manage departments, faculty accounts (with initials and aliases), terms and holidays; bulk import from CSV/XLSX.	Admin, HOD
FR-03	Upload a timetable, preview parsed slots and unresolved initials, and confirm.	HOD, Admin
FR-04	Generate weekly class events for the term, skipping holidays.	System
FR-05	Upload a circular photo, scan or PDF and extract meeting details by OCR.	Faculty, HOD
FR-06	Review, correct, confirm or reject extracted circulars.	HOD
FR-07	View the personal calendar; create, move and cancel events with clash warnings.	Faculty, HOD
FR-08	Find common free slots for a group of faculty.	HOD
FR-09	Subscribe to a private ICS calendar feed and rotate its secret token.	Faculty
FR-10	Set reminder lead times, channels and quiet hours; register for web push.	Faculty
FR-11	Send due reminders exactly once by e-mail and web push.	Scheduler
FR-12	Create personal tasks; assign tasks to a person, a group or the department from templates.	Faculty, HOD
FR-13	Rank tasks by priority, alert before deadlines, send daily overdue alerts and HOD escalations.	System
FR-14	Show the HOD a dashboard of completion rate and overdue tasks per faculty member.	HOD

3.4 Non-Functional Requirements

Table 3: Non-functional requirements

Quality	**Requirement and how it is met**
Correctness	All times stored in UTC and shown in India Standard Time; business rules covered by automated tests.
Reliability	Reminders survive worker crashes (stale-send recovery) and are never duplicated (unique idempotency key).
Security	Argon2id password hashing, 15-minute access tokens, rotating refresh tokens with reuse detection, role checks, rate-limited log-in.
Safety of uploads	File type checked from its first bytes, 15 MB size limit, image pixel limit, PDF page limit.
Usability	Mobile-first web app that installs on a phone; works with existing calendar apps through ICS.
Maintainability	Four-layer architecture, one class per use case, strict type checking.
Deployability	Runs on one department server with Docker Compose; no paid service needed.

3.5 Scope and Constraints

- Three human roles: Admin, HOD and Faculty. A background scheduler acts as a fourth, non-human actor.
- Scanned timetable PDFs without a text layer are rejected with guidance; scanned circulars are read with OCR.
- The system does not create timetables; it reads timetables that the department has already prepared.

4. Object Basics and Project Overview

4.1 Object Basics

Object-oriented analysis and design views a system as a set of objects that work together. Each object holds data and the operations that act on that data. The table below defines each basic concept and shows where it appears in FacultyDesk.

Table 4: Object basics with project examples

Concept	**Meaning**	**Example in FacultyDesk**
Object	A thing with identity, state and behaviour.	One uploaded meeting circular, with its text, extracted fields and status.
Class	A template that defines the attributes and operations of similar objects.	`Circular` defines `ocr_text`, `confidence`, `status` and `confirm()`, `reject()`.
Encapsulation	Data is changed only through the object's own operations.	A `ScheduledNotification` changes status only through `mark_sending()`, `mark_sent()`, `mark_failed()`, `retry()` and `cancel()`; any illegal change raises an error.
Abstraction	Showing what an object does and hiding how.	`UnitOfWork` and the repository interfaces describe what storage must do, not how the database does it.
Inheritance	A class reuses and specialises another class.	`XlsxTimetableParser`, `CsvTimetableParser` and `PdfTimetableParser` inherit from `TimetableParser`.
Polymorphism	The same call behaves differently depending on the object.	`parser.parse(content)` reads Excel, CSV or PDF; `notifier.send(...)` sends an e-mail or a push message.
Value object	An immutable object defined only by its value.	`Initials` (2 to 5 capital letters), `TimeSlot`, `QuietHours`.
Association	A structural link between classes.	A `FacultyProfile` belongs to a `Department`.
Composition	A whole that owns its parts; parts cannot exist alone.	An `Event` owns its `EventAttendee` objects; an `AcademicTerm` owns its `Holiday` objects.

4.2 System Overview

Figure 1 shows the overall structure. Users work in a web application in the browser or on the phone. Requests reach the FastAPI server through a Caddy reverse proxy. Slow work (reading timetables, OCR, sending reminders) runs in Celery workers, and Celery Beat starts periodic jobs. Data lives in PostgreSQL; uploaded files and backups live in MinIO.

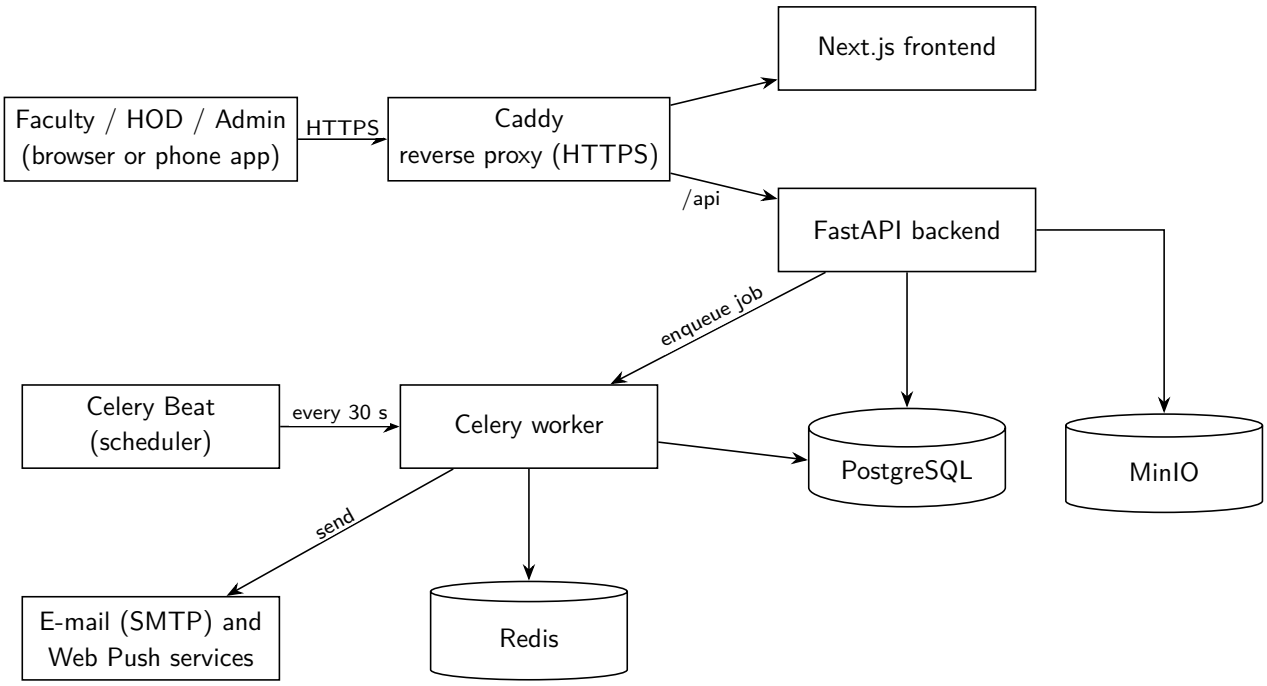


Figure 1: High-level architecture of FacultyDesk

4.3 Project Modules

The system is divided into eight modules. Each module has its own use-case classes, its own set of API endpoints and its own screens, and all modules share one domain model.

Module 1: Identity and Access

Handles log-in, log-out and password change. Passwords are hashed with Argon2id. A short-lived access token (15 minutes) is kept in memory, and a refresh token (7 days) is kept in an httpOnly cookie and replaced on every use; if an old refresh token is presented again, the whole token family is revoked. One `AuthorisationPolicy` class decides what each role may do. Log-in attempts are rate-limited per IP address and per account.

Main classes: `LoginUseCase`, `RefreshTokenUseCase`, `LogoutUseCase`, `AuthorisationPolicy`, `Principal`.

Module 2: Directory and Academic Calendar

Maintains departments, faculty accounts with their initials and aliases, academic terms and holidays. Faculty can be imported in bulk from CSV or Excel with a row-by-row validation report.

Main classes: `CreateFacultyUseCase`, `UpdateFacultyUseCase`, `BulkImportFacultyUseCase`, `ManageTermsUseCase`.

Module 3: Timetable Ingestion

Accepts the department timetable, chooses a parser from the file's first bytes, expands merged period cells, splits co-teacher cells such as `KKS/RNS`, and resolves each initials token in three tiers (exact initials, alias, fuzzy match). The HOD sees a preview and the unresolved tokens before confirming.

Main classes: `UploadTimetableUseCase`, `ParseTimetableUseCase`, `ConfirmTimetableUseCase`, `TimetableParserFactory`, `InitialsResolver`.

Module 4: Circular Scanning

Reads meeting circulars. Images are cleaned (grey scale, noise removal, thresholding, deskewing) and read by Tesseract. The extractor finds date, time, venue, title, agenda and attendees, and a review gate decides whether the circular may be confirmed automatically or needs the HOD.

Main classes: `UploadCircularUseCase`, `ProcessCircularUseCase`, `ConfirmCircularUseCase`, `RejectCircularUseCase`, `OcrEngine`, `CircularExtractor`.

Module 5: Calendar

Keeps every class, meeting and deadline as an `Event` with attendees. It detects clashes with a sweep-line algorithm, finds common free slots, publishes the ICS feed, and every night extends class events 21 days ahead.

Main classes: `CreateEventUseCase`, `UpdateEventUseCase`, `SweepLineClashDetector`, `FreeSlotFinder`, `GenerateIcsFeedUseCase`, `MaterialiseTimetableEventsUseCase`.

Module 6: Reminders and Notifications

Plans one `ScheduledNotification` per attendee, lead time and channel, shifts reminders out of quiet hours, and sends due reminders every 30 seconds by e-mail and web push.

Main classes: `ReminderPlanner`, `ReminderDispatcher`, `CompositeNotifier`, `EmailNotifier`, `WebPushNotifier`.

Module 7: Tasks and Escalation

Manages personal and assigned documentation tasks with templates (CO–PO mapping, IA marks upload, attendance shortage report, lab manual update, course file submission). Tasks are ranked by a priority score; alerts go out 72 hours, 24 hours and 2 hours before the deadline and daily after it, and overdue work is escalated to the HOD.

Main classes: `CreateTaskUseCase`, `AssignTaskUseCase`, `CompleteTaskUseCase`, `ReopenTaskUseCase`, `GetHodDashboardUseCase`.

Module 8: Operations

Checks every upload for type, size and decompression attacks, backs up the database every night to object storage with 14-day retention, and reports reminder delivery metrics to the Admin.

Main classes: `BackupService`, upload guard, metrics endpoints.

4.4 Business Aspects

Stakeholders and Benefits

Table 5: Stakeholders and benefits

Stakeholder	**Need**	**Benefit from FacultyDesk**
Faculty member	Know every class, meeting and deadline on time.	One calendar, reminders on phone and e-mail, clash warnings.
HOD	Distribute the timetable and meetings; track documentation.	Upload once for everyone; dashboard of completion and overdue work.
Department office	Avoid retyping schedules and chasing people.	Works directly from the existing timetable file and circular.
College management	Accreditation work (NBA, NAAC) done on time.	Record of assigned and completed documentation tasks.

Business Process: Before and After

Table 6: Business process comparison

Activity	**Before (manual)**	**With FacultyDesk**
Timetable revision	New sheet circulated; each teacher re-reads it.	HOD uploads the file once; everyone's classes regenerate.
Meeting circular	Paper notice or chat photo.	Photo uploaded, OCR proposes the meeting, HOD confirms, attendees are reminded.

Documentation deadline	Verbal or e-mail reminder.	Task with priority, alerts at 72 h, 24 h and 2 h, daily overdue alerts, HOD escalation.
Clash between duties	Found on the day.	Shown when the event is created; free-slot finder suggests another time.

Feasibility

- **Technical:** built entirely with mature open-source components; runs on one server with Docker Compose.
- **Economic:** no licence cost. Running cost is one server (a department machine or a cloud virtual machine) and an SMTP relay for e-mail; web push uses the browser vendors' free push services. Hardware cost and time saved were not measured; they need a pilot in a department.
- **Operational:** faculty keep using the documents they already produce; the app installs on phones and exports to existing calendar apps.

Risks and Controls

Table 7: Risks and controls

Risk	**Control in the design**
Wrong meeting from a bad OCR read	Review gate: document confidence $\geq$ 70, date and time confidence $\geq$ 0.85, every attendee resolved, no duplicate; otherwise the HOD reviews.
Class assigned to the wrong teacher	Exact initials first, then alias, then fuzzy match only above similarity 0.85 with a 0.10 margin; ambiguous tokens are reported, never guessed.
Duplicate or lost reminders	Unique idempotency key per (target, user, offset, channel); row locking with `SKIP LOCKED`; recovery of stuck sends.
Exposure of personal data	Hashed passwords, short-lived tokens, role checks, private ICS links that can be rotated.
Data loss	Nightly compressed database backup with 14-day retention and a tested restore.

5. UML Modelling

The Unified Modeling Language (UML) is the standard visual language for specifying, constructing and documenting the parts of an object-oriented system [1], [2]. This section models FacultyDesk with the main UML diagrams. For each diagram it gives the theory and notation, the diagram for the project, an explanation, supporting tables and a short conclusion. All class, method, state and attribute names in the diagrams are taken from the project's source code.

Table 8: UML diagrams presented in this report

Diagram	**Kind**	**What it shows for FacultyDesk**
Use case diagram	Behavioural	Actors and the services the system gives them
Class diagram	Structural	Domain classes, attributes, operations and relations
Sequence diagram	Interaction	Time order of messages in three key flows
Collaboration diagram	Interaction	Objects and numbered messages when a circular is confirmed
Statechart diagram	Behavioural	Life cycles of Circular, ScheduledNotification and TimetableUpload
Activity diagram	Behavioural	Workflow of timetable ingestion across three swimlanes
Package diagram	Structural	Code packages and their dependencies
Component diagram	Structural	Runtime components and the interfaces between them
Deployment diagram	Structural	Containers on the department server and the network links
Entity–relationship diagram	Data model	Database tables, keys and relations

5.1 Use Case Diagram

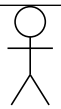
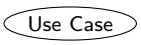
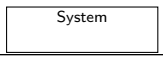
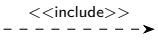
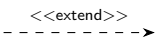
Theory

A use case diagram is a behavioural UML diagram that gives a high-level picture of *what* a system offers to its users, without describing *how* each function is built. It contains the system boundary, the actors outside it, the use cases inside it, and the relationships among them.

- **Actor:** an external role that interacts with the system to reach a goal. A *primary actor* starts the use case; a *supporting actor* helps the system deliver it. An actor may be a person or another system, such as a scheduler.
- **Use case:** a unit of functionality that gives an observable result of value to an actor, named with a verb phrase such as "Upload timetable".
- **System boundary:** a rectangle that encloses the use cases and defines the scope.

Use Case Notation

Table 9: Use case diagram notation

Element	Symbol	Meaning
Actor		A role played by a user or an external system.
Use case		A function the system performs for an actor.
System boundary		Encloses the use cases and defines the scope.
Association		The actor takes part in the use case.
Include		The base use case always uses the included one.
Extend		Optional behaviour added to a base use case under a condition.
Generalization		The child is a specialised form of the parent.

Explanation

- **Actors:** *Admin* manages terms, holidays and the faculty directory. *HOD* uploads and confirms timetables, reviews circulars and assigns tasks. *Faculty* upload circulars, view their calendar, manage tasks, set reminder preferences and subscribe to the ICS feed. *Scheduler* is the background job runner (Celery Beat), a non-human actor.
- **Upload timetable** <<include>> **Resolve faculty initials**, because a timetable can never be stored without knowing which teacher each cell refers to.
- **Confirm timetable** <<include>> **Generate weekly class events**, because confirmation always creates the classes.

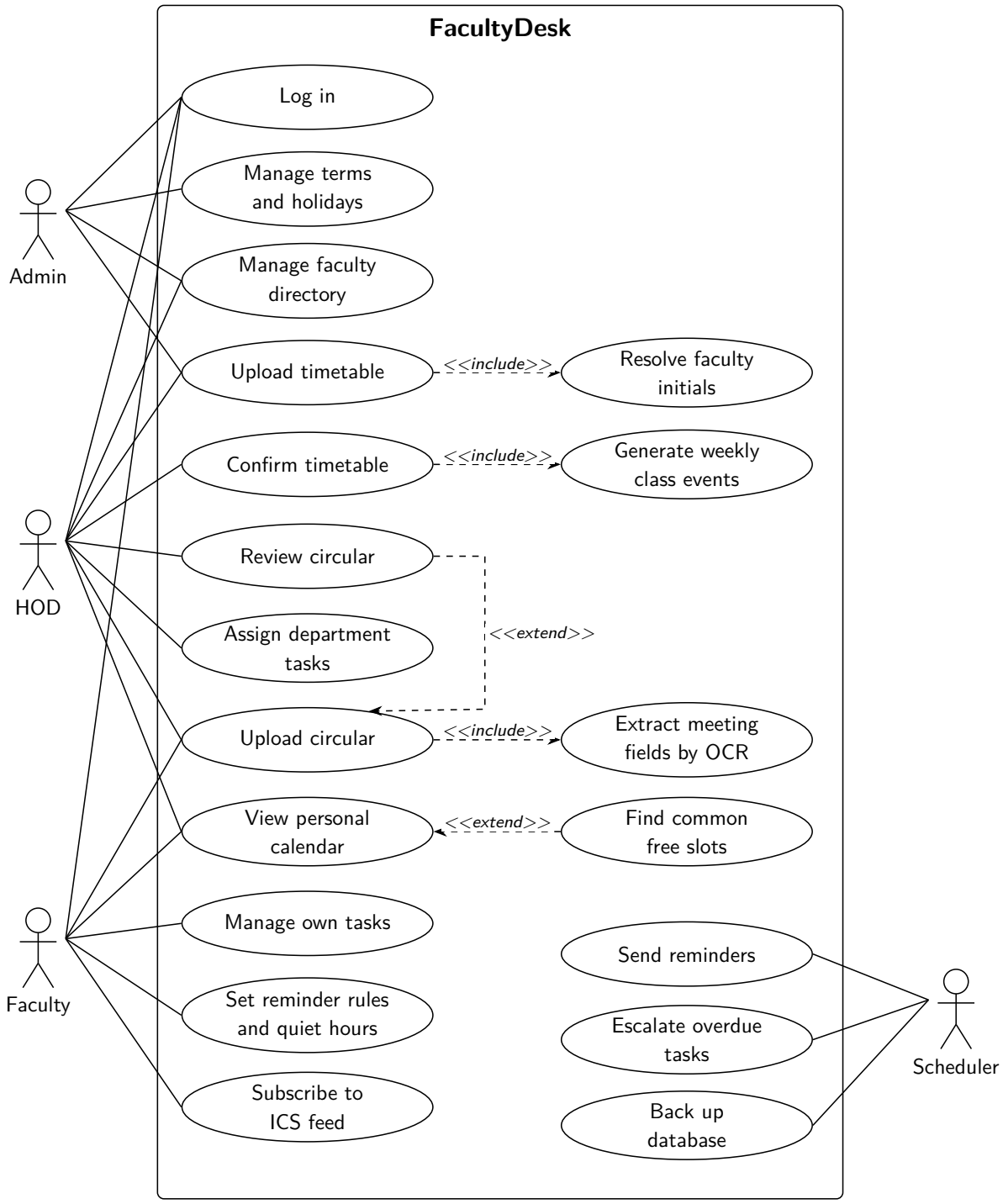


Figure 2: FacultyDesk use case diagram

- **Upload circular** <<include>> **Extract meeting fields by OCR**, because every uploaded circular is read.
- **Review circular** <<extend>> **Upload circular**: review is needed only when the extraction fails the confidence gate.
- **Find common free slots** <<extend>> **View personal calendar**: it is an optional step when planning a meeting.
- The Scheduler sends reminders, escalates overdue tasks and backs up the database without any human action.

Actors and Use Cases Identified

Table 10: Actors and use cases

Actor	**Type**	**Use cases**
Admin	Primary	Log in, Manage terms and holidays, Manage faculty directory, Upload timetable
HOD	Primary	Log in, Manage faculty directory, Upload timetable, Confirm timetable, Review circular, Assign department tasks, Upload circular, View personal calendar
Faculty	Primary	Log in, Upload circular, View personal calendar, Manage own tasks, Set reminder rules and quiet hours, Subscribe to ICS feed
Scheduler	Supporting (system)	Send reminders, Escalate overdue tasks, Back up database

Table 11: Use case scenario: Upload and Confirm Timetable

Use Case ID	UC-01
Use Case Name	Upload and Confirm Timetable
Actors	HOD (primary)
Precondition	HOD is logged in; the academic term exists; faculty profiles with initials exist in the department.
Main Flow	1. HOD selects the term and uploads the timetable file (xlsx, csv or pdf, at most 15 MB). 2. System checks the file type from its first bytes, stores it and starts parsing. 3. System parses the grid, expands merged period cells and resolves each initials token (*include: Resolve faculty initials*). 4. System shows the preview slots and any unresolved tokens. 5. HOD confirms the preview. 6. System saves courses, sections, rooms and slots, and generates the weekly class events (*include: Generate weekly class events*).
Alternative Flow	2a. Unsupported file or scanned PDF without text: the upload is rejected with guidance. 4a. Some initials are unresolved or ambiguous: HOD adds an alias in the directory and uploads again.
Postcondition	Every teacher named in the timetable sees the term's classes on the calendar, without holidays, with reminders planned.

Table 12: Use case scenario: Upload and Confirm Meeting Circular

Use Case ID	UC-02
Use Case Name	Upload and Confirm Meeting Circular
Actors	Faculty or HOD (upload), HOD (review)
Precondition	User is logged in; department faculty have initials and aliases.
Main Flow	1. User uploads a photo, scan or PDF of the circular. 2. System stores it and replies at once; a worker processes it. 3. System reads the text (PDF text layer, or image clean-up and Tesseract OCR) (*include: Extract meeting fields by OCR*). 4. System extracts date, time, venue, title, agenda and attendees. 5. The circular fails the gate and enters the review queue (*extend: Review circular*). 6. HOD corrects any field and confirms. 7. System creates a MEETING event for every attendee and plans their reminders.
Alternative Flow	3a. File or OCR error: circular becomes FAILED and can be processed again. 5a. All fields pass the gate and the department allows automatic confirmation: the meeting is created without review. 6a. HOD rejects the circular: it becomes REJECTED and no event is created.
Postcondition	Each named attendee sees the meeting and receives reminders at their chosen lead times.

Table 13: Use case scenario: Send Reminders

Use Case ID	UC-03
Use Case Name	Send Reminders
Actors	Scheduler (primary), Faculty (receiver)
Precondition	Pending reminders exist with a fire time in the past; the user has an e-mail address or a push subscription.
Main Flow	1. Every 30 seconds the Scheduler starts the dispatch job. 2. System claims up to 100 due reminders, locking them so no other worker takes them. 3. For each reminder, the system builds the message from the event or task. 4. The reminder is sent by e-mail or web push and marked SENT.

Alternative Flow	3a. The task is already completed: the reminder is cancelled. 4a. Sending fails: the reminder is retried later, up to five attempts, then marked FAILED.
Postcondition	Each reminder is delivered at most once; the Admin can see delivery counts and failures.

Table 14: Use case scenario: Assign and Complete Documentation Task

Use Case ID	UC-04
Use Case Name	Assign and Complete Documentation Task
Actors	HOD (assign), Faculty (complete)
Precondition	HOD and faculty accounts exist; task templates are seeded.
Main Flow	1. HOD picks a template (for example IA marks upload) or writes a task, sets a deadline and importance. 2. HOD assigns it to one teacher, a group or the whole department. 3. System creates one task per assignee, a DEADLINE event on each calendar, and alerts at 72 h, 24 h and 2 h before the deadline. 4. Faculty see the task ranked by priority and mark it complete. 5. System cancels the remaining alerts and updates the HOD dashboard.
Alternative Flow	4a. Deadline passes: daily overdue alerts start and the HOD receives an escalation summary. 5a. Faculty reopen the task: future alerts are scheduled again.
Postcondition	The HOD dashboard shows the completion rate and overdue count per teacher.

Conclusion

The use case diagram captures every service of FacultyDesk and who uses it. The <<include>> relation models mandatory steps such as initials resolution and OCR, and <<extend>> models optional steps such as review and free-slot finding.

5.2 Class Diagram

Theory

A class diagram is a structural UML diagram that shows the classes of a system, their attributes and operations, and the relationships between them. It is the main diagram of object-oriented design and maps directly to code. A class is drawn as a rectangle with three compartments: name, attributes and operations. Visibility is shown with **+** public, **–** private, **#** protected and **~** package.

Relationships Between Classes

Table 15: Class relationship notation

Relationship	**Symbol**	**Example in FacultyDesk**
Association	——	FacultyProfile belongs to a Department.
Aggregation	◇——	CompositeNotifier has an EmailNotifier and a WebPushNotifier.
Composition	◆——	Event owns its EventAttendees; AcademicTerm owns its Holidays.
Generalization	——▷	XlsxTimetableParser is a TimetableParser.
Realization	- - - - - - - - ▷	SystemClock implements the Clock protocol.
Dependency	- - - - - - - - - ➤	A Circular creates an Event when confirmed.

Multiplicity

Notation	**Meaning**
1	Exactly one
0..1	Zero or one
* or 0..*	Zero or more
1..*	One or more

Table 16: Multiplicity notation

Classes Identified

Table 17: Domain classes with attributes and operations

Class	**Attributes**	**Operations / rules**
User	email, password_hash, role, is_active, quiet hours	quiet_hours()

FacultyProfile	full_name, initials, aliases, email, phone, ics_token	initials must be 2–5 capital letters
Department	code, name	–
AcademicTerm	name, start_date, end_date, time-zone	owns Holidays
Holiday	date, label	–
TimetableUpload	file_key, status, parse_report	start_parsing(), mark_preview_ready(), confirm(), fail()
TimetableSlot	time_slot, course, section, room, faculty	–
Event	kind, title, start_utc, end_utc, loca-tion, source	reschedule(), cancel(), add_attendee()
EventAttendee	event_id, faculty_id, response	–
Circular	file_key, ocr_text, extracted_fields, confidence, status	start_processing(), mark_needs_review(), confirm(), reject(), fail()
ScheduledNotification	channel, fire_at_utc, idempo-tency_key, attempts, status	mark_sending(), mark_sent(), mark_failed(), retry(), cancel()
ReminderRule	event_kind, offsets_minutes, chan-nels	offsets cannot be negative
Task	title, deadline_utc, importance (1–5), category, status	update_deadline(), change_status()

Relationships

- **FacultyProfile** is the central class: it belongs to a **Department**, is linked to one **User** account, teaches **TimetableSlot**s and attends **Event**s through **EventAttendee**.
- **Composition:** a **TimetableUpload** owns its slots, an **Event** owns its attendees, and an **AcademicTerm** owns its holidays; the parts have no meaning without the whole.
- **Event** is one class for classes, meetings and deadlines; its `kind` attribute tells them apart, so clash detection, reminders and the ICS feed treat them the same way.
- A confirmed **TimetableSlot** is *materialised* into weekly CLASS events, and a confirmed **Circular** *creates* one MEETING event.
- A **ScheduledNotification** targets an **Event** or a **Task**; a **Task** has one assignee **User**.

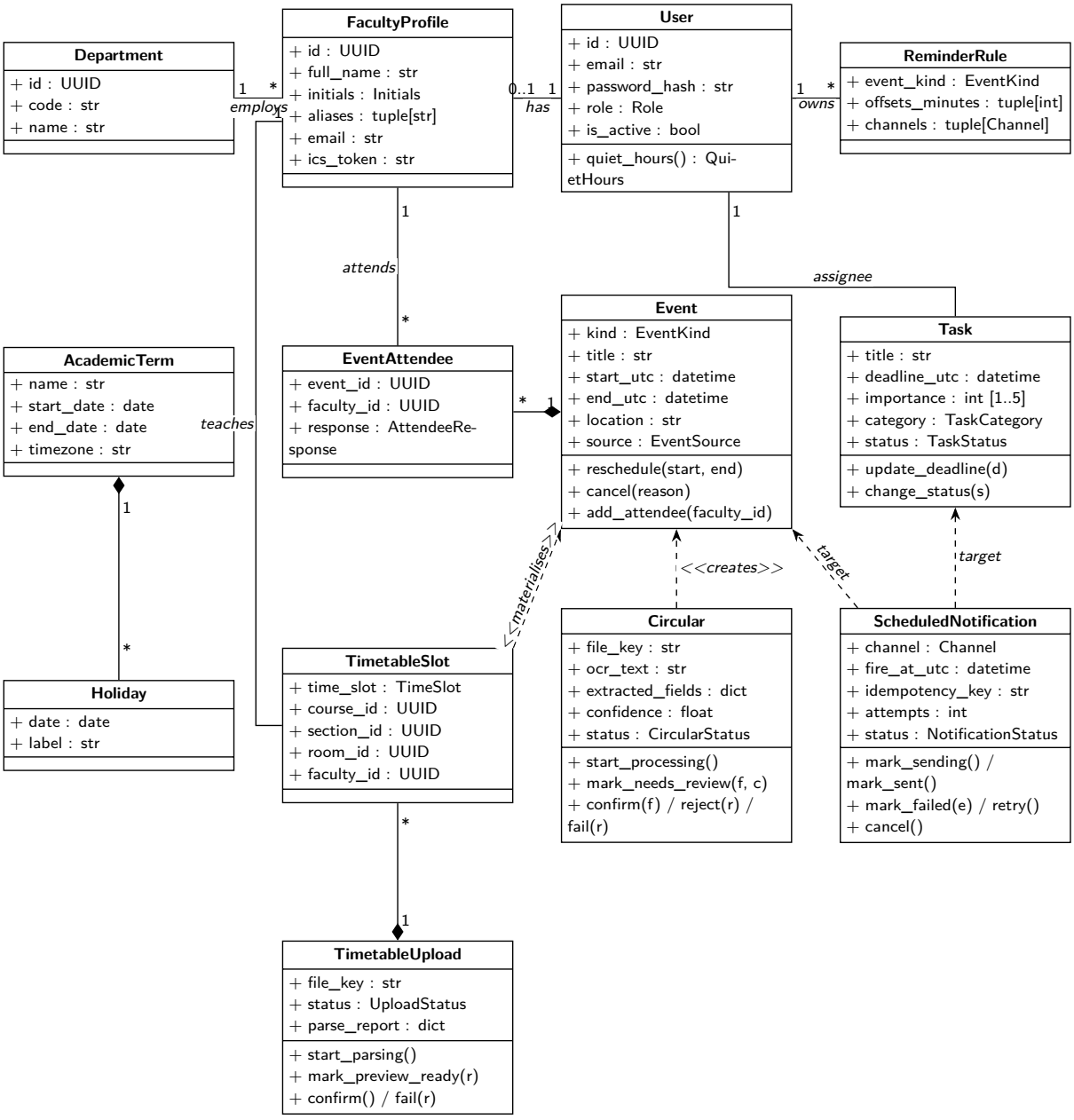


Figure 3: FacultyDesk domain class diagram

Conclusion

The class diagram shows that the business rules live in the classes themselves: each class validates its own data and controls its own status changes, which is the basis of the object-oriented design.

5.3 Sequence Diagrams

Theory

A sequence diagram is an interaction diagram that shows how objects communicate by exchanging messages in time order. Time flows from top to bottom. Each participant has a lifeline; messages are arrows between lifelines; combined fragments such as `alt` (alternatives) and `loop` (repetition) group messages.

Sequence Diagram Notation

Table 18: Sequence diagram notation

Element	**Symbol**	**Meaning**
Lifeline	:Object	An object taking part in the interaction.
Synchronous message	——▶	The sender waits for the reply.
Asynchronous message	——➤	The sender continues without waiting.
Reply message	◁- - - - - - - - -	Return of control or a value.
Self message	–	A message an object sends to itself.
alt / loop fragment	–	Alternative paths with guards / repeated messages.

Explanation

1. The HOD uploads the file (1). The router passes it to `UploadTimetableUseCase`, which stores it and returns a `TimetableUpload` in state UPLOADED (2–4).
2. The router hands parsing to a Celery worker and replies `202 Accepted` at once (5–6), so the web request stays short.
3. The worker runs `ParseTimetableUseCase`: the factory picks the parser and the initials are resolved (7–10).
4. The HOD reads the preview (11–12) and confirms (13). `ConfirmTimetableUseCase` saves the slots and raises a `TimetableConfirmed` event (14–17).

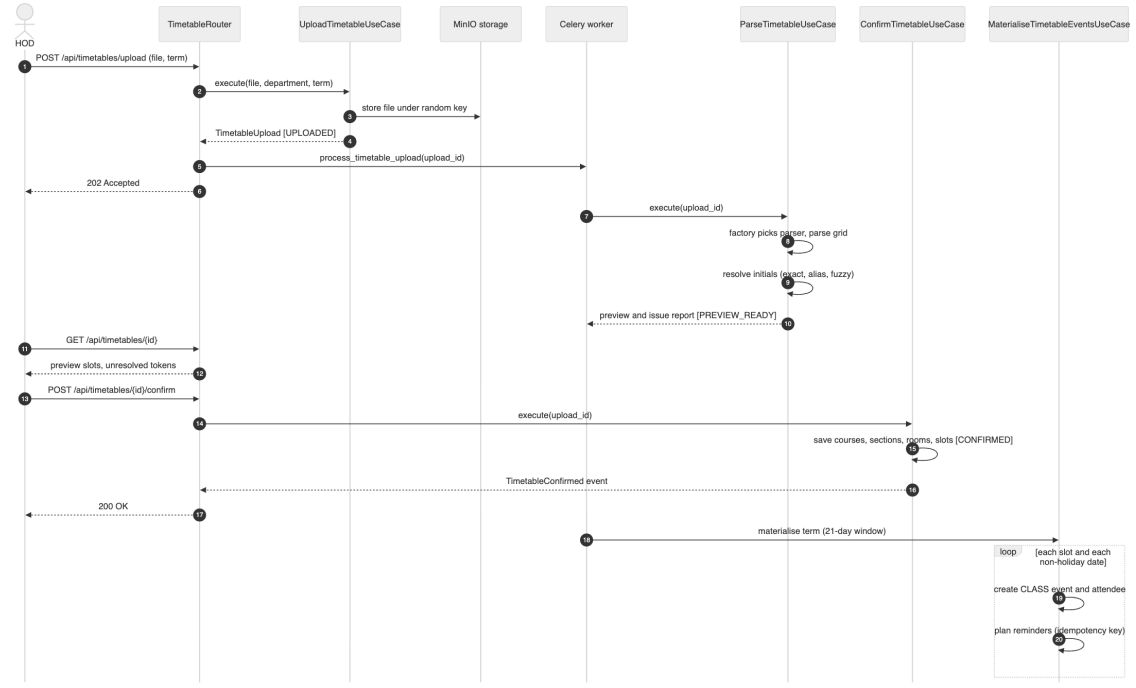


Figure 4: Sequence diagram: timetable upload and class generation

5. The worker then expands each slot over the term in a `loop`, creating CLASS events and planning reminders (18–20).

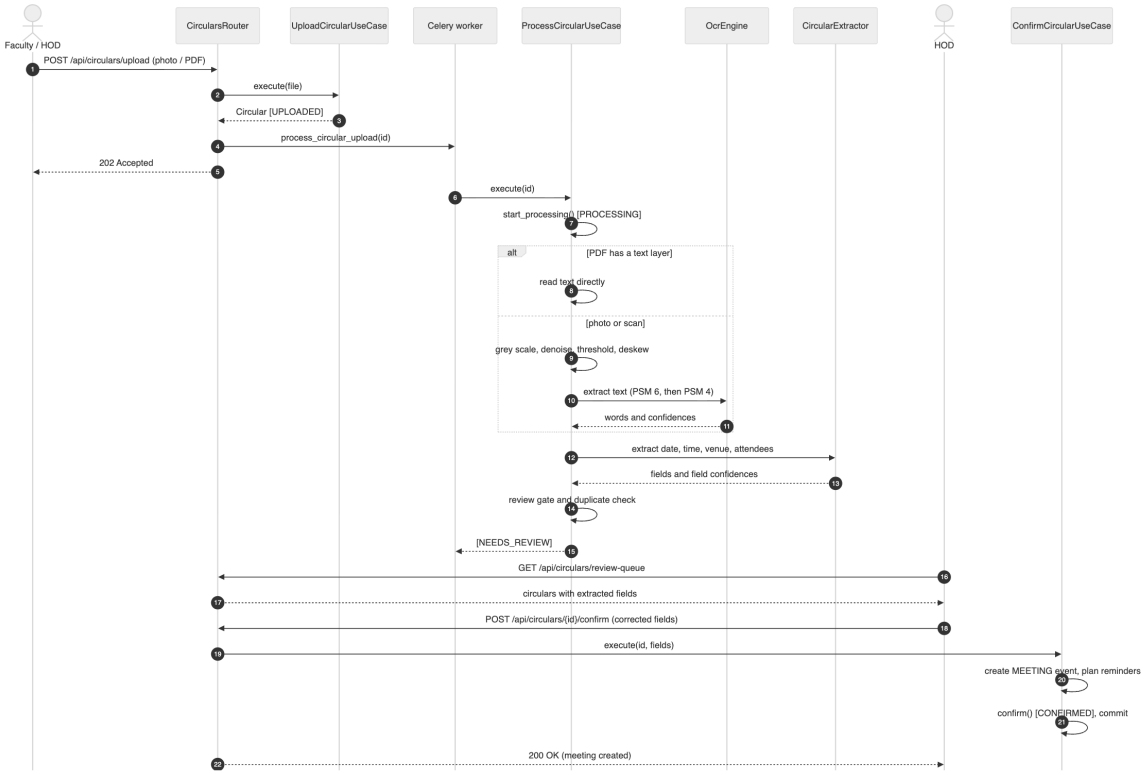


Figure 5: Sequence diagram: circular upload, OCR, review and confirmation

Explanation

1. Upload is answered immediately (1–5); processing runs in a worker (6–7).
2. The `alt` fragment shows the two reading paths: a PDF with a text layer is read directly (8); a photo or scan is cleaned and read by `OcrEngine` with two page-segmentation modes (9–11).
3. `CircularExtractor` returns the fields with a confidence for each (12–13), and the review gate puts the circular in NEEDS_REVIEW (14–15).
4. The HOD opens the review queue, corrects the fields and confirms (16–19); `ConfirmCircularUseCase` creates the meeting, plans reminders and commits (20–22).

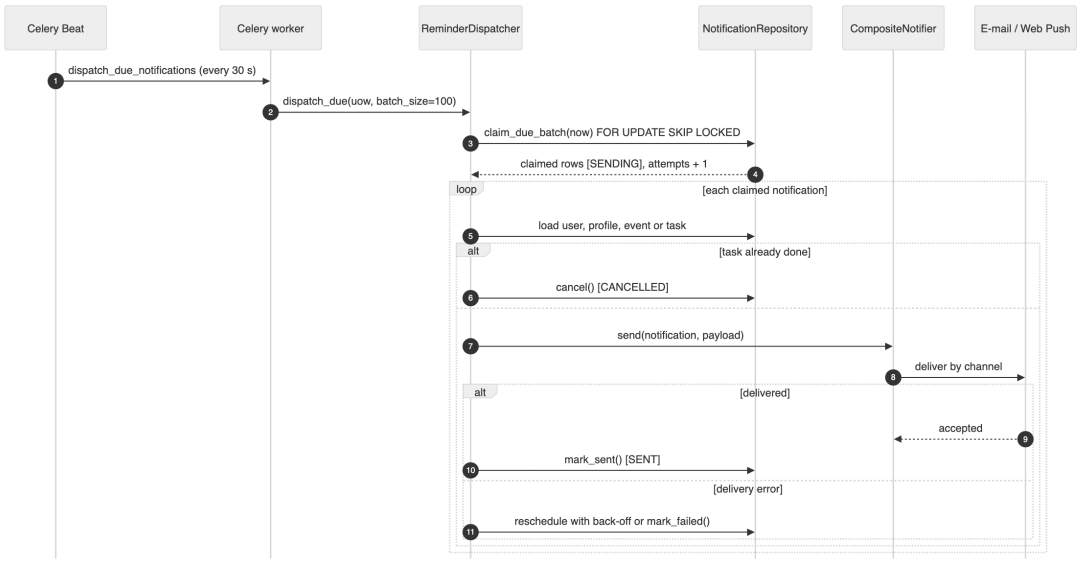


Figure 6: Sequence diagram: periodic reminder dispatch

Explanation

1. Every 30 seconds Celery Beat starts the dispatch job (1–2).
2. The dispatcher claims up to 100 due reminders with `SELECT ... FOR UPDATE SKIP LOCKED`, so two workers can never claim the same row, and marks them SENDING (3–4).
3. For each reminder it loads the user and the event or task (5). If the task is already done, the reminder is cancelled; otherwise it is sent through `CompositeNotifier`, which chooses e-mail or web push (6–8).
4. A successful send is marked SENT; a failed one is rescheduled with back-off or marked FAILED after five attempts (9–11).

Conclusion

The three sequence diagrams show the time order of the main flows and the design decision behind them: slow work is moved to workers so that users always get an immediate answer.

5.4 Collaboration (Communication) Diagram

Theory

A collaboration diagram, called a communication diagram in UML 2, shows the same interaction as a sequence diagram but arranges the objects by their links instead of by time. Messages are written along the links and numbered in nested decimal form (1, 1.1, 1.1.1), so the order and the call nesting are still visible.

Table 19: Sequence versus collaboration diagram

Point	**Sequence diagram**	**Collaboration diagram**
Emphasis	Time order of messages	Structural links between objects
Layout	Lifelines from left to right, time downwards	Objects placed freely, joined by links
Message order	Vertical position	Decimal numbers (1, 1.1, 1.1.1)
Best for	Long flows with many steps	Seeing which objects work together

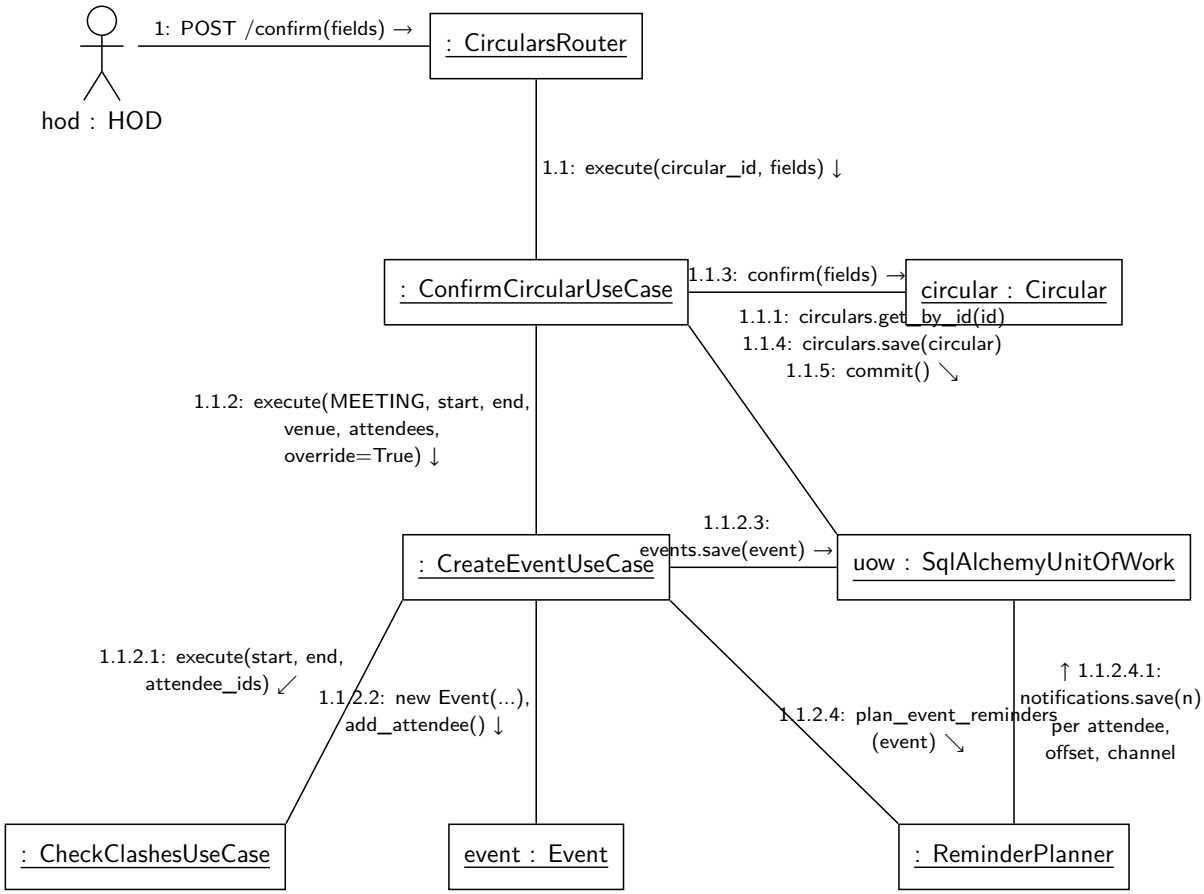


Figure 7: Collaboration diagram: HOD confirms a meeting circular

Messages

Table 20: Messages in the collaboration diagram

No.	From	To	Message and purpose
1	HOD	CircularsRouter	POST /confirm with the corrected fields
1.1	CircularsRouter	ConfirmCircularUseCase	execute(circular_id, fields)
1.1.1	ConfirmCircularUseCase	uow (circulars)	load the circular
1.1.2	ConfirmCircularUseCase	CreateEventUseCase	create a MEETING with clash override
1.1.2.1	CreateEventUseCase	CheckClashesUseCase	find clashes for all attendees
1.1.2.2	CreateEventUseCase	event : Event	build the event and add attendees
1.1.2.3	CreateEventUseCase	uow (events)	save the event
1.1.2.4	CreateEventUseCase	ReminderPlanner	plan reminders for every attendee
1.1.2.4.1	ReminderPlanner	uow (notifications)	save one notification per attendee, offset, channel
1.1.3	ConfirmCircularUseCase	circular : Circular	confirm(fields): status becomes CONFIRMED
1.1.4–1.1.5	ConfirmCircularUseCase	uow	save the circular and commit once

Conclusion

The collaboration diagram shows that one confirmation touches eight objects, and that every write goes through the same unit of work, so the circular, the meeting, its attendees and its reminders are stored together or not at all. The clash check runs but is overridden, because the HOD has already reviewed the meeting.

5.5 Statechart Diagrams

Theory

A statechart (state machine) diagram describes the states an object passes through during its life and the events that move it from one state to another. It is most useful for objects whose behaviour depends on their current state. In FacultyDesk three classes keep their allowed transitions in a table in the code and raise `InvalidTransition` for any other move.

Statechart Notation

Table 21: Statechart notation

Element	Symbol	Meaning
Initial state	●	Starting point of the object's life.
State	State	A condition in which the object waits or does work.
Transition	event [guard] / action →	Change of state caused by an event.
Final state	◉	End of the object's life.

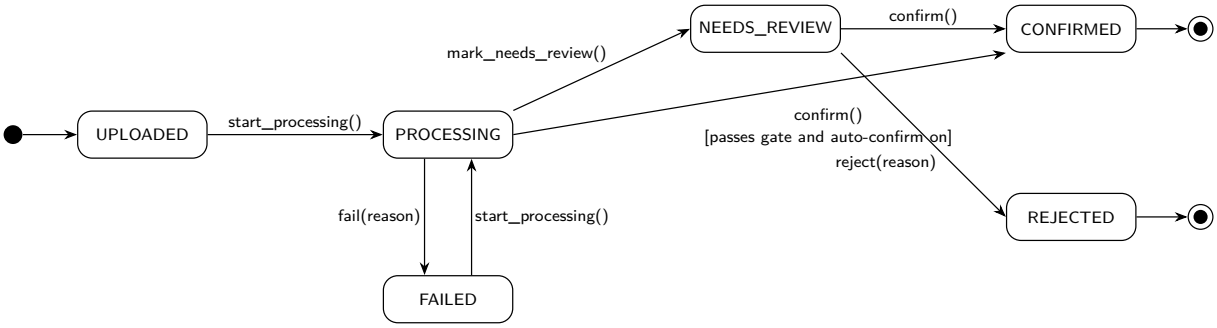


Figure 8: Statechart diagram of Circular

Table 22: State transition table: Circular

Current state	Event	Next state
Initial	File stored	UPLOADED
UPLOADED	start_processing()	PROCESSING
PROCESSING	mark_needs_review() (gate failed)	NEEDS_REVIEW
PROCESSING	confirm() (gate passed, auto-confirm on)	CONFIRMED
PROCESSING	fail(reason) (file or OCR error)	FAILED
FAILED	start_processing() (retry)	PROCESSING
NEEDS_REVIEW	confirm() by HOD	CONFIRMED
NEEDS_REVIEW	reject(reason) by HOD	REJECTED

Table 23: State transition table: ScheduledNotification

Current state	Event	Next state
Initial	Reminder planned	PENDING
PENDING	mark_sending() (claimed by dispatcher; attempts + 1)	SENDING
PENDING	cancel() (event cancelled or task done)	CANCELLED
SENDING	mark_sent()	SENT
SENDING	mark_failed(error)	FAILED
SENDING	stale recovery (more than 5 minutes)	PENDING
FAILED	retry() while attempts < 5	PENDING

Table 24: State transition table: TimetableUpload

Current state	Event	Next state
UPLOADED	start_parsing()	PARSING
PARSING	mark_preview_ready(report)	PREVIEW_READY
PREVIEW_READY	confirm() by HOD	CONFIRMED
any non-final state	fail(reason)	FAILED

Conclusion

The statecharts make the life cycle of each important object explicit. Because the same tables are enforced in code, a circular can never be confirmed twice and a reminder can never be sent from the wrong state.

5.6 Activity Diagram

Theory

An activity diagram shows the flow of control from one action to the next, similar to a flowchart, with support for decisions, parallel work and swimlanes that show who performs each action. It is used to model business processes and workflows.

Activity Diagram Notation

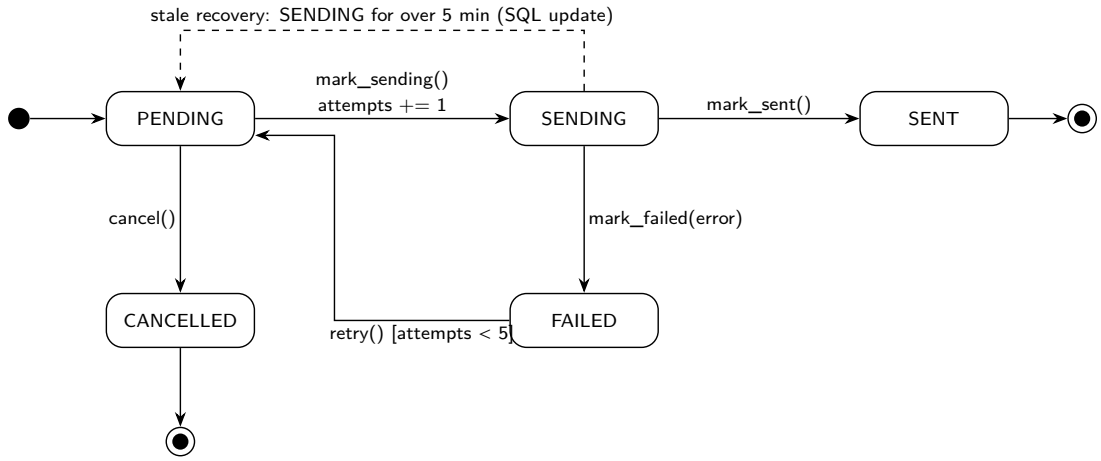


Figure 9: Statechart diagram of ScheduledNotification

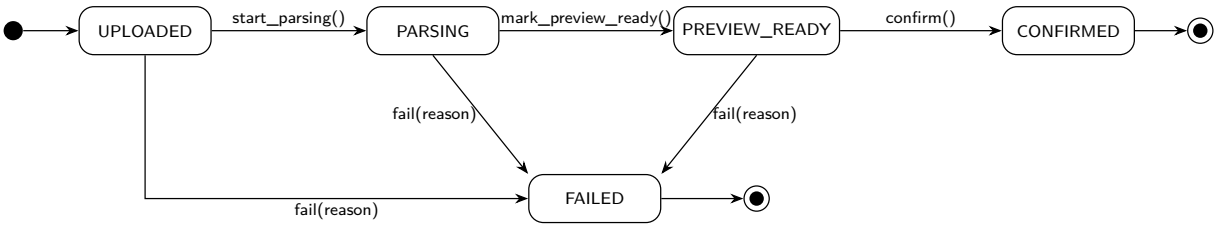


Figure 10: Statechart diagram of TimetableUpload

Table 25: Activity diagram notation

Element	Symbol	Meaning
Initial node	●	Start of the activity.
Action	Action	A single step of work.
Decision / merge	◇	Choose one path based on a guard, or merge paths.
Fork / join	▬▬	Split into, or synchronise, parallel flows.
Control flow	⟶	Order in which actions execute.
Swimlane	–	Partition showing who performs the actions.
Final node	◉	End of the activity.

Explanation

- **Swimlanes:** the HOD, the API server and the Celery worker each perform their own actions.
- **First decision:** an invalid file is rejected before anything is stored.
- **Worker steps:** the factory chooses the parser; cells are split into course, section, room and initials; co-teacher cells are split; initials are resolved in three tiers.
- **Second decision and loop:** if the preview shows issues, the HOD fixes aliases or the file and uploads again; only a clean preview is confirmed.
- **After confirmation:** weekly recurrence is expanded over the term, holidays are skipped, CLASS events are created and reminders are planned.

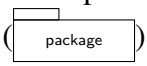
8	Review preview; fix issues if any	HOD	Decision (loop to 1)
9	Save courses, sections, rooms, slots	API server	Action
10	Expand recurrence, create events, plan reminders	Worker	Action

Conclusion

The activity diagram models the complete ingestion workflow, including the validation stop, the fix-and-retry loop and the division of work between the HOD, the API and the background worker.

5.7 Package Diagram

Theory

A package diagram groups model elements or code into packages and shows the dependencies between them. A package is drawn as a folder (

); a dependency is a dashed arrow, often labelled <<import>> or <<use>>. It is used to check that the layering of a system is respected.

Explanation

- The **frontend** calls the backend only through the REST API described by an OpenAPI contract.
- **api** and **workers** call **application**; **workers** (and `api/deps.py`) wire the concrete adapters from **infrastructure**.
- **infrastructure** implements the repository interfaces declared in **domain**.
- **domain** imports nothing outside itself; we checked this by scanning every import in the backend.
- The arrow labelled *direct import* marks four application files that import infrastructure classes (OCR engine and extractor, parser factory and initials resolver, notifier base class, MinIO storage). Moving these behind domain interfaces would complete the layering.

Conclusion

The package diagram confirms that the business core (`domain`) is independent of frameworks and shows the one place where the layering is not yet complete.

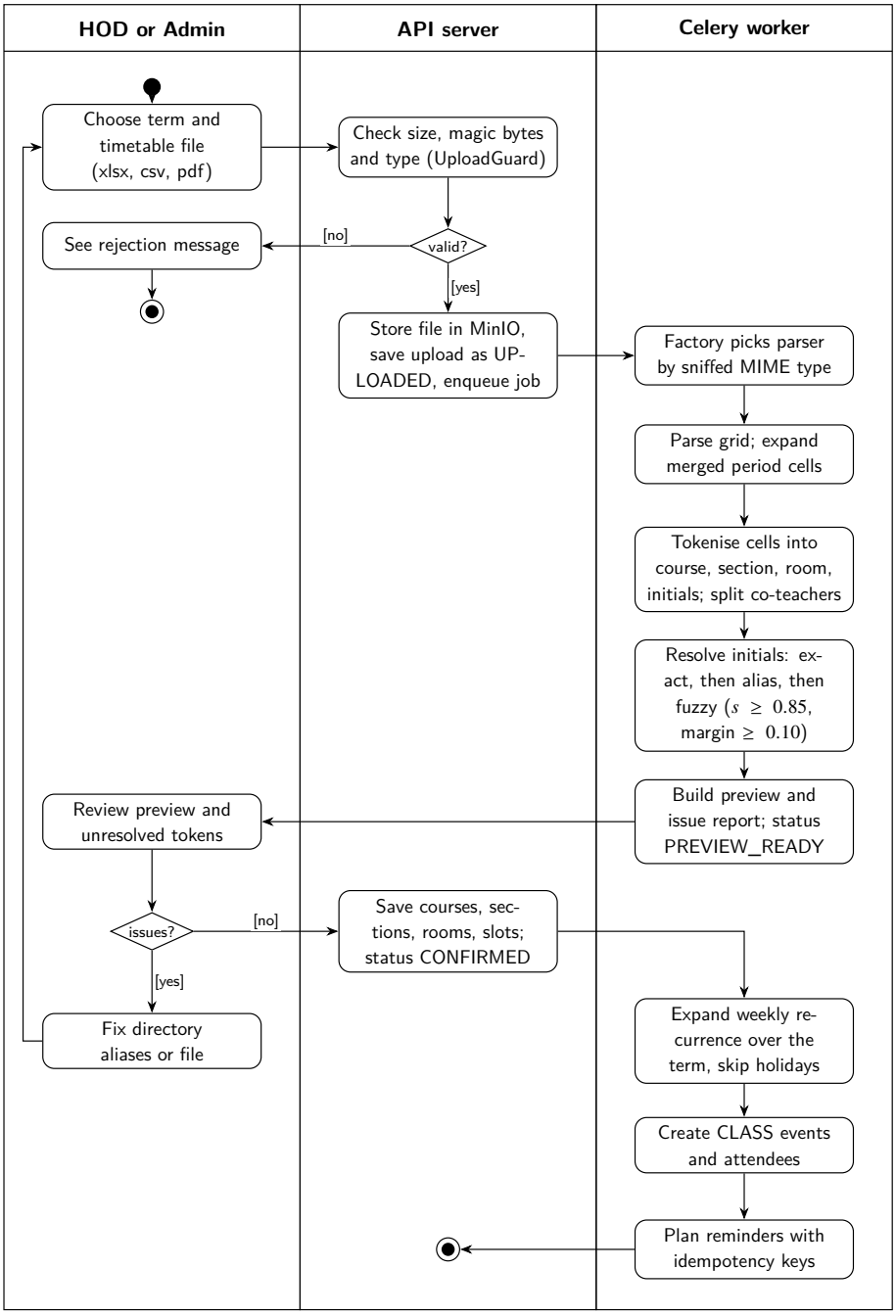


Figure 11: Activity diagram: timetable ingestion and initials resolution

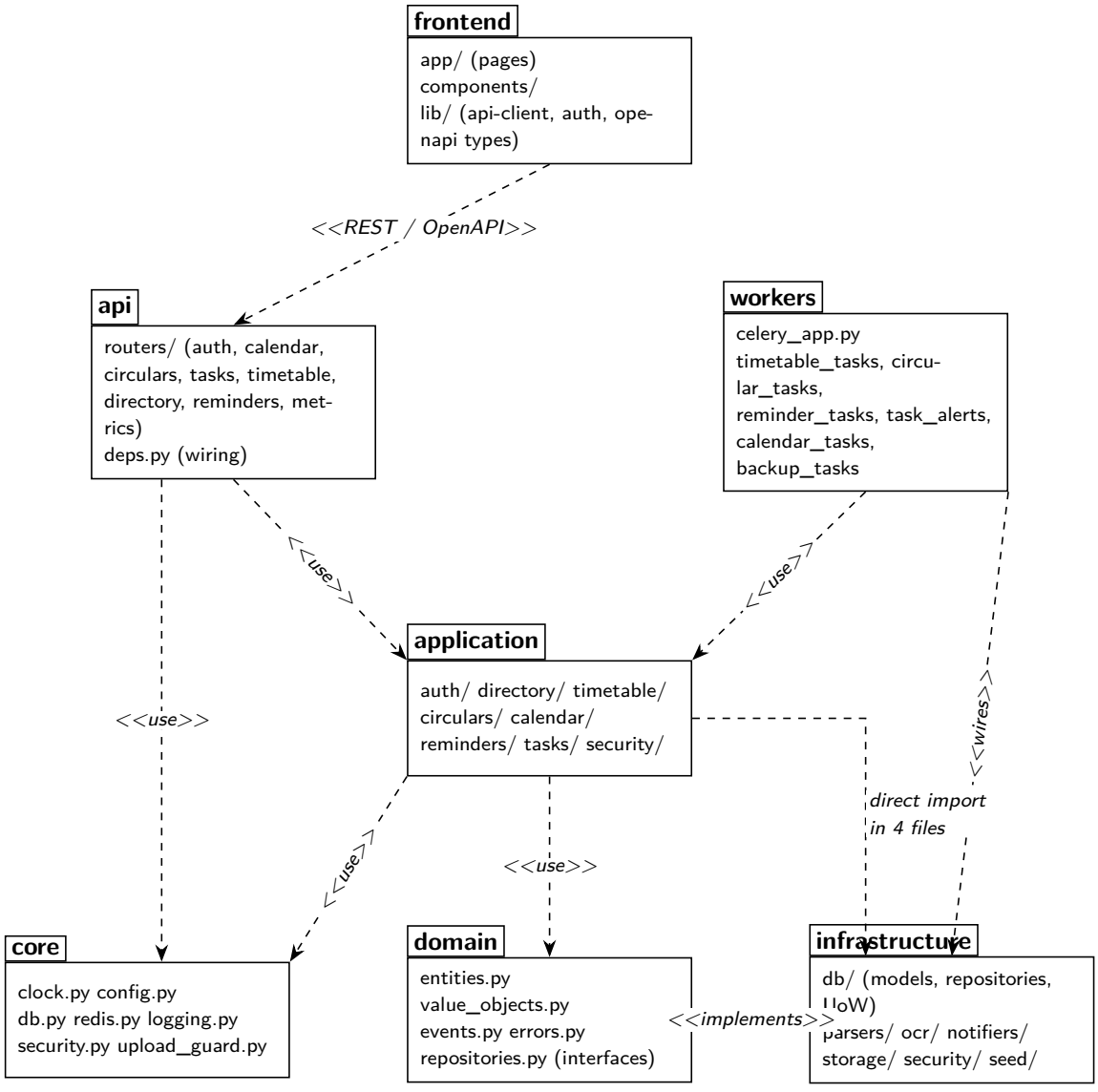


Figure 12: Package diagram with import dependencies

5.8 Component Diagram

Theory

A component diagram shows the replaceable parts of a running system and the interfaces through which they connect. A component (<<component>> Name) provides interfaces, drawn as a ball (——◯), and requires interfaces from other components.

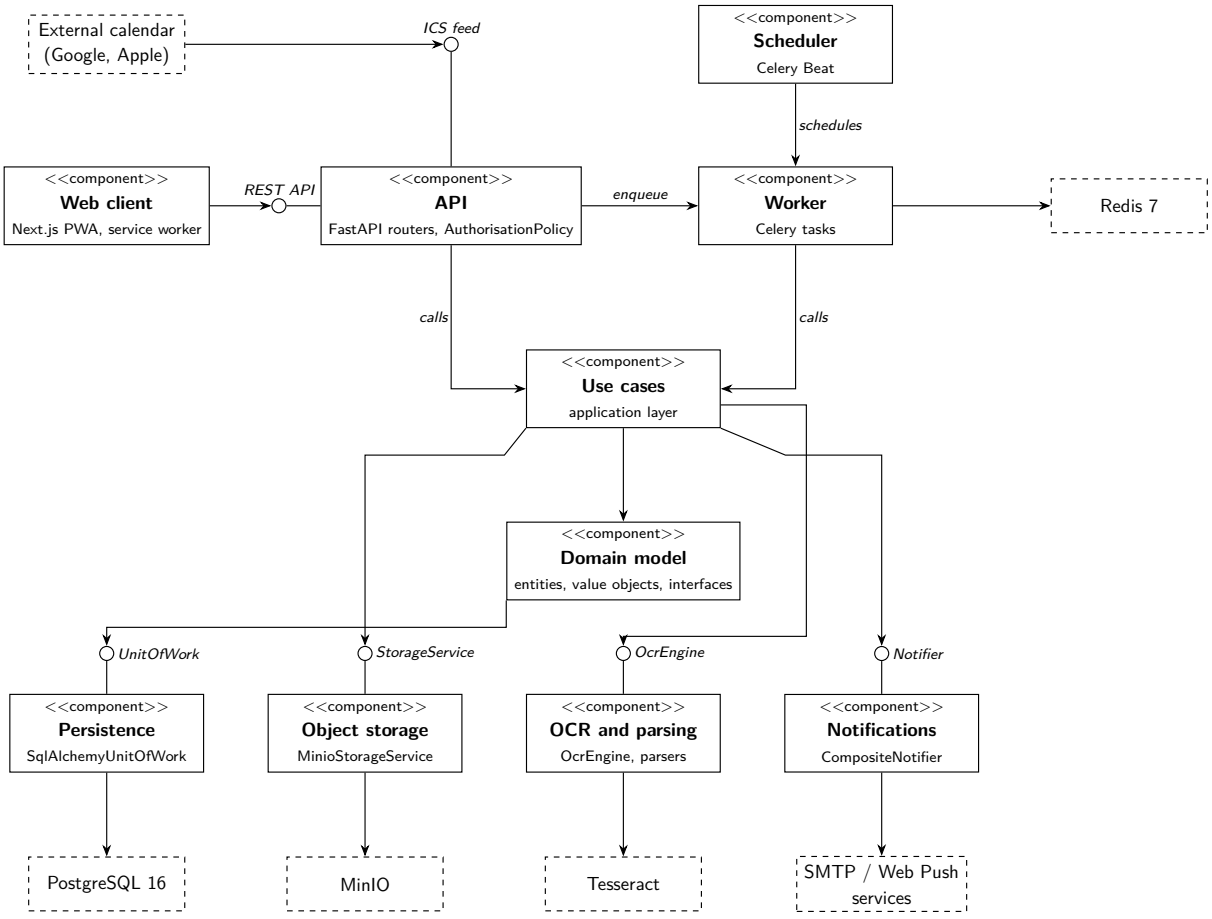


Figure 13: Component diagram with provided interfaces

Components Identified

Table 27: Components and interfaces

Component	Responsibility	Interface
Web client	Screens for all roles; offline cache and push handler	requires REST API
API	Routers, validation, role checks	provides REST API, ICS feed
Worker / Scheduler	Background jobs and periodic schedule	uses Redis queue

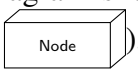
Use cases	One class per user goal	requires UnitOfWork, StorageService, OcrEngine, Notifier
Domain model	Entities, value objects, interfaces	declares repository interfaces
Persistence	SQLAlchemy repositories and unit of work	provides UnitOfWork
Object storage	File upload and download in MinIO	provides StorageService
OCR and parsing	Tesseract OCR, timetable parsers	provides OcrEngine
Notifications	E-mail and web push	provides Notifier

Conclusion

The component diagram shows that the API and the worker share the same use-case component, and that every external service is reached through one adapter, so it can be replaced without changing the use cases. `UnitOfWork` and `Notifier` are abstract types in the code; `OcrEngine` and `StorageService` are concrete classes today.

5.9 Deployment Diagram

Theory

A deployment diagram shows the physical nodes (devices and execution environments, drawn as 3-D boxes 

), the artifacts deployed on them, and the communication paths between nodes.

Explanation

- All server parts run as containers on one machine under Docker Compose; application containers run as a non-root user.
- Browsers, the installed phone app and external calendar apps reach only the Caddy container over HTTPS. Caddy sends `/api` requests to the backend and all other requests to the frontend.
- The worker and the scheduler share PostgreSQL, Redis and MinIO with the backend; the worker delivers e-mail and web push to outside services.
- In *college-LAN mode* Caddy uses an internal certificate, so the system can run inside the campus network without a public domain.

Conclusion

The deployment diagram shows that the complete system fits on one department server and needs no paid service.

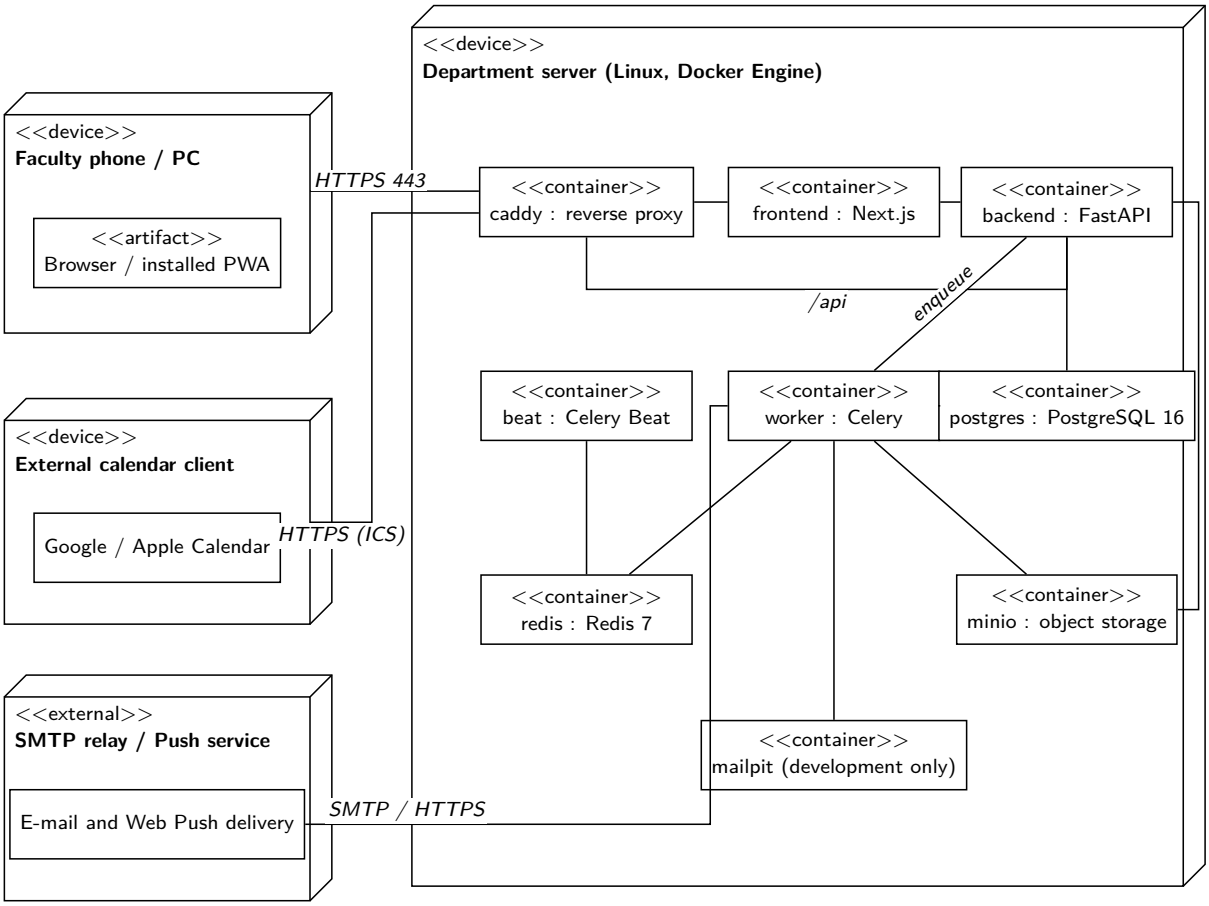


Figure 14: Deployment diagram (production on a department server)

5.10 Entity–Relationship Diagram

The ER diagram in Figure 15 is generated by a script directly from the database model definitions, so its 20 tables, columns and keys match the database exactly. All timestamps are stored as `timestamptz` in UTC. Unique keys enforce business rules in the database itself, for example one `idempotency_key` per reminder and one `ics_token` per faculty member. Because the schema is wide, the diagram is rotated on the page.

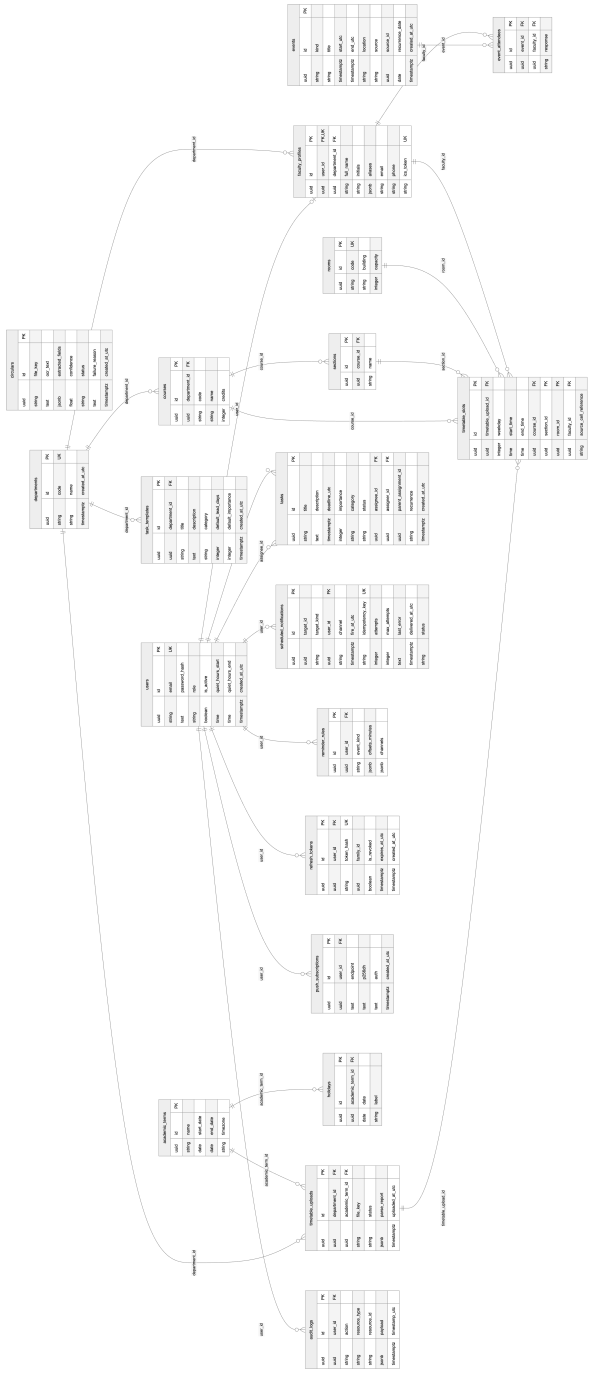


Figure 15: Entity–relationship diagram of the PostgreSQL schema

6. Object-Oriented Analysis, Design & Architecture

6.1 Object-Oriented Analysis

Object-oriented analysis finds the classes of the problem domain before any design decision is made. We used the noun analysis method [2]: mark the nouns of the problem statement, classify them, filter out the ones that are not classes, and add classes that domain knowledge requires.

Step 1: Potential Objects (Nouns and Noun Phrases)

From the problem statement in Section 3: department, system, calendar, faculty member, HOD, timetable, timetable file, cell, initials, teacher, class, term, holiday, photo, meeting circular, date, time, venue, name, meeting, details, reminder, e-mail, phone notification, quiet hours, documentation task, group, deadline, work, alert.

Step 2: Classification

Table 28: Classification of candidate objects

Category	**Candidates**
People and roles	faculty member, teacher, HOD, group
Organisations	department
Things and documents	timetable file, cell, photo, meeting circular, e-mail, phone notification
Concepts	calendar, initials, quiet hours, details, term
Events	class, meeting, holiday, deadline, reminder, alert, documentation task
Attributes	date, time, venue, name

Step 3: Filtering

Table 29: Filtering of candidates

Candidate	**Decision**	**Reason**
system	Removed	It is the system itself, not a domain object.
teacher, faculty member	Merged → FacultyProfile	Same person; profile holds initials and aliases.
HOD, Admin	Role of User	They differ only in permissions (`Role` value).
class, meeting, deadline	Merged → Event	One interval type with a `kind`; clash checks and reminders treat them alike.

reminder, alert	Merged → ScheduledNotification	One planned send with its own life cycle.
date, time, venue, name	Attributes	Properties of Event or FacultyProfile.
initials, quiet hours	Value objects	Defined only by their value; must validate themselves.
cell	Merged → TimetableSlot	A parsed cell is one weekly slot.
e-mail, phone notification	Channel value	Delivery channel of a notification.
calendar, details, group	Removed	A view over events, too vague, or a set of profiles.

Step 4: Classes Added from Domain Knowledge

- **User** – the log-in account, separate from the faculty profile (an Admin may have no profile).
- **TimetableUpload** – one uploaded file with its parse report and status.
- **EventAttendee** – links a faculty member to an event with a response.
- **ReminderRule** – a user's lead times and channels for each kind of event.
- **AcademicTerm** – the term dates and time zone that bound class generation.

Step 5: Final Domain Classes

Department, User, FacultyProfile, AcademicTerm, Holiday, Course, Section, Room, TimetableUpload, TimetableSlot, Event, EventAttendee, Circular, ScheduledNotification, ReminderRule, PushSubscription, Task, TaskTemplate, AuditLog, RefreshToken; value objects Initials, TimeSlot and QuietHours.

6.2 Object-Oriented Design

Layered (Clean) Architecture

The backend is divided into four layers [5]. Dependencies point inwards, towards the domain.

Use cases receive their collaborators through their constructors (dependency injection). The same `ProcessCircularUseCase` therefore runs in a worker in production and in a test with a test database, and the same `ReminderDispatcher` runs with `SystemClock` in production and

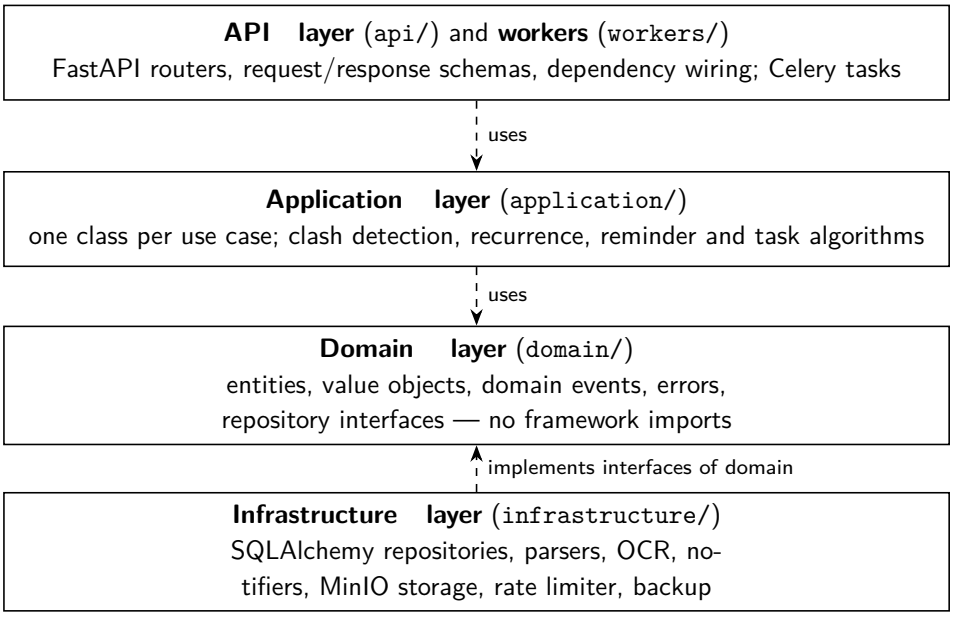


Figure 16: Layered architecture of the backend

with `FrozenClock` in tests. As shown in the package diagram, four application files still import infrastructure classes directly; the domain layer has no outward imports at all.

Design Principles Applied

Table 30: Design principles

Principle	**How FacultyDesk applies it**
Single responsibility	Each use-case class serves one goal; `InitialsResolver` only resolves initials; `CircularExtractor` only extracts fields.
Open–closed	A new timetable format needs a new `TimetableParser` subclass and one line in the factory; a new channel (such as SMS) needs a new `Notifier`.
Liskov substitution	Any `Notifier` can be given to the dispatcher; tests substitute frozen-time and test-database objects.
Interface segregation	Eighteen small repository interfaces, one per aggregate, instead of one large data-access class.
Dependency inversion	Use cases depend on `UnitOfWork`, `Notifier` and `Clock` abstractions, not on SQLAlchemy or SMTP.
Information expert (GRASP)	`QuietHours` decides whether a time is quiet; `Circular` decides which status change is legal; `Task` validates its own importance and deadline.

Design Patterns

The patterns follow the catalogues of Gamma *et al.* [3] and Fowler [4]. Figures 17 and 18 show the classes involved.

Table 31: Design patterns used

Pattern	Classes	Problem it solves
Repository	EventRepository, TaskRepository and 16 others	Hides SQL behind collection-like interfaces so domain code stays storage-free.
Unit of Work	UnitOfWork, SqlAlchemyUnitOfWork	Saves a circular, its meeting, attendees and reminders in one transaction.
Strategy	TimetableParser subclasses; Notifier subclasses	Swaps the parsing algorithm by file type and the delivery method by channel.
Factory	TimetableParserFactory	Chooses the parser from the file's first bytes, not from its name.
Composite	CompositeNotifier	Lets the dispatcher send through e-mail and push as if they were one notifier.
State	Circular, ScheduledNotification, TimetableUpload	Encodes legal life cycles as transition tables; illegal moves raise errors.
Domain events	EventScheduled, EventRescheduled, EventCancelled, TaskDeadlineChanged, TimetableConfirmed	Records what happened to an entity; reminder re-planning reacts to rescheduling and cancellation.

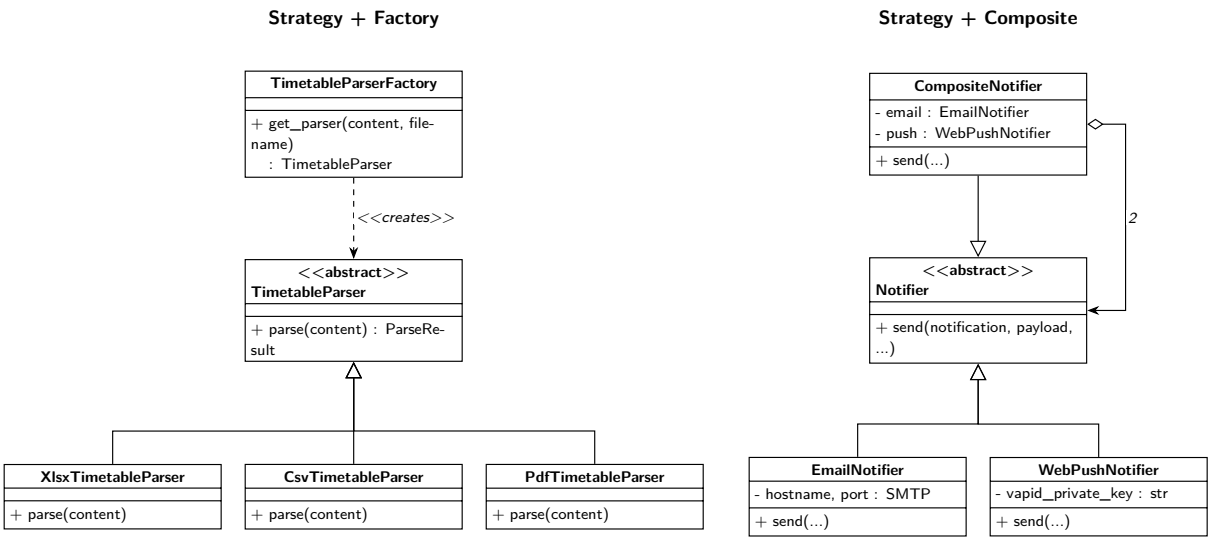


Figure 17: Class diagram of the Strategy, Factory and Composite patterns

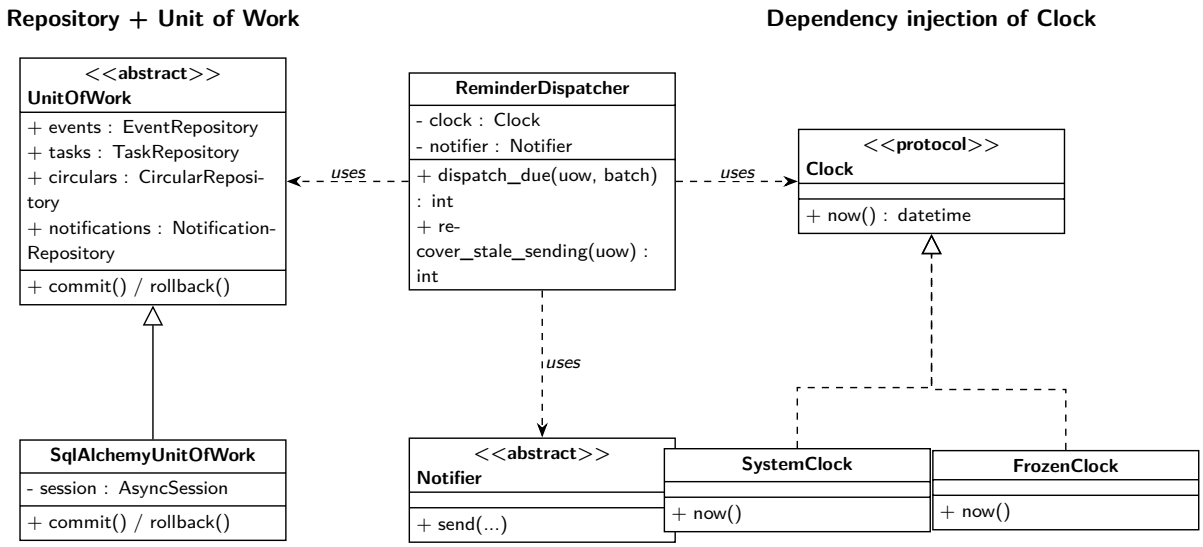


Figure 18: Class diagram of the Repository and Unit of Work patterns and the injected Clock

6.3 Architecture Decisions

Table 32: Architecture decisions

Decision	**Reason**
Background workers for slow work	OCR and timetable parsing take seconds; the API stores the file, replies `202 Accepted` and a Celery worker does the work, so every web request stays short.
UTC storage, local expansion	Every timestamp is stored in UTC. Weekly recurrence is expanded in local time (Asia/Kolkata) and then converted, so a 9:00 class stays at 9:00.
Injected clock	No code reads the system time directly; an injected `Clock` makes every time rule testable.
Exactly-once reminders	Key $k = \text{SHA256}(\text{target} \,\|\| \text{user} \,\|\| \text{offset} \,\|\| \text{channel})$ with a unique database constraint; rows claimed with `FOR UPDATE SKIP LOCKED`.
Human in the loop	No OCR result reaches a calendar unless it passes the review gate and auto-confirmation is enabled; otherwise the HOD confirms it.
One authorisation policy	One `AuthorisationPolicy` class decides what Admin, HOD and Faculty may do, so permission rules live in one place.

7. Implementation (Algorithms Pseudo Code)

7.1 Implementation Overview

The backend has 76 Python source files and the frontend has eight main screens. Table 33 maps each module to its main source files. The algorithms below are written from the code; each names the file it comes from. All times are in UTC unless marked local.

Table 33: Modules and source files

Module	**Main source files (backend/app/)**	**Screen**
Identity and access	application/auth/*, application/security/policy.py, infrastructure/security/rate_limiter.py	/login
Directory	application/directory/*, api/routers/directory.py	/directory
Timetable	application/timetable/*, infrastructure/parsers/*	/timetables
Circulars	application/circulars/use_cases.py, infrastructure/ocr/*	/circulars
Calendar	application/calendar/* (clash_detector, free_slots, recurrence, ics_service)	/calendar, /today
Reminders	application/reminders/* (planner, dispatcher, maths), infrastructure/notifiers/*	/settings
Tasks	application/tasks/*, workers/task_alerts.py	/tasks
Operations	core/upload_guard.py, infrastructure/storage/backup_service.py	–

7.2 Algorithm 1: Three-Tier Initials Resolution

A timetable cell such as `CN (KKS)` names the teacher by initials. The resolver tries an exact match, then an alias, and only then a fuzzy match based on the Levenshtein edit distance [7]. A fuzzy match is accepted only if it is clearly better than the second-best candidate, so the system never guesses between two similar teachers.

Algorithm 1 Three-tier initials resolution (infrastructure/parsers/initials_resolver.py)

Require: token t, faculty profiles F; $\theta_{sim} = 0.85$, $\theta_{margin} = 0.10$

$t \leftarrow \textsc{Upper}(\textsc{Trim}(t))$
$E \leftarrow \{f \in F : f.\text{initials} = t\}$ ▷ tier 1: exact initials
if $|E| = 1$ **then return** $\textsc{Resolved}(E_1)$
else if $|E| > 1$ **then return** Ambiguous
end if
$A \leftarrow \{f \in F : t \in f.\text{aliases}\}$ ▷ tier 2: exact alias
if $|A| = 1$ **then return** $\textsc{Resolved}(A_1)$
else if $|A| > 1$ **then return** Ambiguous
end if
for all $f \in F$ **do** ▷ tier 3: normalised similarity
    $s_f \leftarrow 1 - \textsc{Lev}(t, f.\text{initials})/\max(|t|, |f.\text{initials}|)$
end for
$s_1, s_2 \leftarrow$ best and second-best scores
if $s_1 \geq \theta_{sim}$ **and** $s_1 - s_2 \geq \theta_{margin}$ **then return** $\textsc{Resolved}(F_1)$
end if
return Unresolved ▷ shown to the HOD

7.3 Algorithm 2: Sweep-Line Clash Detection

Two events clash if their half-open intervals overlap: $\max(s_1, s_2) < \min(e_1, e_2)$. Events that only touch (one ends at 10:00, the next starts at 10:00) do not clash. Sorting by start time and keeping the currently open events in a min-heap finds all clashes in $O(n \log n + k)$ time for k clashing pairs, instead of comparing every pair in $O(n^2)$.

Algorithm 2 Sweep-line clash detection (application/calendar/clash_detector.py)

Require: intervals $I = \{[s_i, e_i)\}$

sort I by (s, e); $H \leftarrow$ empty min-heap keyed by end time; $C \leftarrow \emptyset$
for all $x \in I$ in sorted order **do**
    while $H \neq \emptyset$ **and** $\textsc{Min}(H).e \leq x.s$ **do**
        $\textsc{Pop}(H)$
    end while
    for all $y \in H$ **do**
        $C \leftarrow C \cup \{(y, x)\}$
    end for
    $\textsc{Push}(H, x)$
end for
return C

7.4 Algorithm 3: Weekly Class Generation

Each confirmed timetable slot is expanded week by week inside the term and the current 21-day window. Holidays are skipped, and times are built in the term's local time zone before conversion to UTC.

Algorithm 3 Class generation for one slot (application/calendar/recurrence.py)

Require: slot (weekday, start, end), term $[d_0, d_1]$, holidays D_h, window $[w_0, w_1]$

$a \leftarrow \max(d_0, w_0)$; $b \leftarrow \min(d_1, w_1)$; **if** $a > b$ **return** $\emptyset$
for all date $d \in \textsc{WeeklyRule}(a, b, \text{weekday})$ **do**
    if $d \notin D_h$ **then**
        $s \leftarrow \text{ToUTC}(d + \text{start})$; $e \leftarrow \text{ToUTC}(d + \text{end})$ ▷ local → UTC
        emit occurrence (d, s, e)
    end if
end for

7.5 Algorithm 4: Reminder Planning with Quiet Hours

For each attendee, lead time and channel, the fire time is the event start minus the lead time. If that falls inside the user's quiet window, the reminder moves to the end of the window, unless the event starts before the window ends (an early-morning class), in which case it is kept. A SHA-256 key makes planning repeatable without duplicates.

Algorithm 4 Reminder planning (application/reminders/planner.py, maths.py)

Require: event E; per-user rule (offsets O, channels K) or the default for E.kind

for all attendee user u, offset $o \in O$, channel $c \in K$ **do**
    $f \leftarrow E.s - o$
    if local time of f lies in u's quiet window $[q_s, q_e)$ **then**
        $w \leftarrow$ end of that window in UTC (next day if the window crosses midnight)
        if $E.s > w$ **then** $f \leftarrow w$
        end if
    end if
    $k \leftarrow \text{SHA256}(E.\text{id} \,\|\, u \,\|\, o \,\|\, c)$
    if no notification with key k exists **then** save a PENDING notification (u, c, f, k)
    end if
end for

7.6 Algorithm 5: Reminder Dispatch

Every 30 seconds the dispatcher claims a batch of due reminders under a row lock that other workers skip, and sends each one. Failed sends are designed to be retried after a random delay up to $\min(1800, 30 \cdot 2^n)$ seconds (full-jitter back-off).

Algorithm 5 Reminder dispatch (application/reminders/dispatcher.py)

transaction: claim up to 100 rows with status PENDING and fire time ≤ now `FOR UPDATE SKIP LOCKED`; mark them SENDING; attempts += 1
for all claimed notification n **do**
  if target task is already done **then** CANCEL(n); **continue**
  end if
  if SEND(n) succeeds **then** MARKSENT(n)
  else
    $\delta \leftarrow \text{UNIFORM}(0, \min(1800, 30 \cdot 2^{\text{attempts}}))$
    if attempts < 5 **then** reschedule n at now $+\delta$
    else MARKFAILED(n)
    end if
  end if
end for

7.7 Algorithm 6: Task Priority Score

Each active task gets a score between 0 and 1 that balances importance and urgency; urgency decays exponentially with the hours left, with a time constant of 48 hours. Overdue tasks score between 1 and 2, so they always rank first, and completed tasks score −1.

Algorithm 6 Task priority score (application/tasks/maths.py)

Require: importance $I \in \{1..5\}$, deadline D, now T; $w_I = 0.4$, $w_U = 0.6$, $\tau = 48$ h
if task is completed **then return** -1
end if
$h \leftarrow (D - T)$ in hours
if $h \le 0$ **then return** $1 + \min(|h|/24, 7)/7$
end if
return $w_I \cdot (I - 1)/4 + w_U \cdot e^{-h/\tau}$

7.8 Algorithm 7: Circular Review Gate

The OCR confidence of a document is the length-weighted mean of its word confidences, $C = \sum \ell_i c_i / \sum \ell_i$, so long words count more than short ones. A circular is confirmed automatically only if every condition below holds.

Algorithm 7 Circular review gate (infrastructure/ocr/extractor.py)

Require: document confidence C, field confidences C_{date}, C_{time}, attendees A

if $C \geq 70$ **and** $C_{date} \geq 0.85$ **and** $C_{time} \geq 0.85$ **and** $|A| \geq 1$ **and** $\forall a \in A : C_a \geq 0.85$ **and not** duplicate **then**
    if department allows automatic confirmation **then return** CONFIRMED
    end if
end if
return NEEDS_REVIEW (with the list of reasons)

8. Results

All results in this section were produced on 13 November 2026 on a laptop (Apple Silicon, macOS, Python 3.12.14) against the running Docker Compose stack. The raw outputs are saved in the project under `report/latex/results/`.

8.1 Automated Backend Tests

The backend test suite was run with `pytest`: **83 tests passed and 0 failed** in 11.1 seconds. Of these, 68 are unit tests of the business rules and algorithms, and 15 are integration tests that run against real PostgreSQL 16 and Redis 7.

Table 34: Backend test results by file

Test file	**Tests**	**What it checks**
unit/test_domain_invariants	10	Entity validation and state-machine transitions
unit/test_hardening	10	File-type sniffing, size, pixel and page limits
unit/test_task_maths	9	Priority score ranges, alert times, completion rate
unit/test_ocr_maths	8	Length-weighted confidence, time grammar, deskew
unit/test_parsers	6	Excel merged cells, CSV, PDF tables, cell tokens
unit/test_initials_matching	4	Exact, alias, fuzzy and ambiguous initials
unit/test_reminder_maths	4	Fire time, idempotency key, back-off, quiet hours
7 other unit files	17	Clash detection, free slots, recurrence, ICS, clock, policy, bulk import
8 integration files	15	Log-in flow, timetable, circular, calendar, task and reminder pipelines, rate limiter, repositories
Total	**83**	**All passed**

8.2 End-to-End Browser Tests

Four user journeys were run in a real Chromium browser with Playwright against the running frontend and API: **4 passed** in 12.9 seconds.

Table 35: End-to-end test results

No.	Journey verified in the browser	Result	Time
1	HOD uploads `standard_timetable.xlsx` and confirms; faculty KKS sees the classes on the calendar	Pass	3.7 s
2	Faculty uploads a circular; HOD reviews and confirms; attendees see the meeting	Pass	5.0 s
3	A scheduled meeting produces a notification record marked SENT	Pass	0.6 s
4	HOD assigns a template task; faculty completes it; pending alerts are resolved	Pass	2.2 s

8.3 Algorithm Outputs on Worked Examples

The values below were computed by calling the project's own functions with the time fixed at 04:30 UTC on 13 November 2026 (script `results/worked_examples.py`).

Task Priority Score

Task	Importance	Hours to deadline	Priority P
IA marks upload	5	6	0.9295
CO–PO mapping	4	48	0.5207
Lab manual update	2	240	0.1040
Course file submission	3	−36 (overdue)	1.2143

Table 36: Priority scores of four sample tasks

The overdue task ranks first, then the urgent high-importance task. For example, IA marks upload scores $0.4 \times 4/4 + 0.6 \times e^{-6/48} = 0.4 + 0.5295 = 0.9295$.

Initials Similarity

Token	Candidate	Similarity	Decision
KKS	KKS	1.000	Match (exact)
PARKAVI	PARKAVY	0.857	Fuzzy match if margin ≥ 0.10
KKS	KSS	0.667	Rejected (below 0.85)
RNS	KKS	0.333	Rejected

Table 37: Normalised Levenshtein similarity examples

Other Worked Examples

- **Quiet hours:** a 10:00 IST class on 14 November with a 12-hour reminder would fire at 22:00 IST, inside a 22:00–07:00 quiet window. The planner moved it to 07:00 IST (01:30 UTC), still before the class.
- **Task alerts:** a task due in 5 days produced alerts exactly 72 hours, 24 hours and 2 hours before the deadline.
- **OCR confidence:** the words "Meeting on 18/11/2026 at 2:30 PM" with confidences 96, 91, 72, 88, 65 and 90 give a length-weighted confidence of $2190/27 = 81.11$, above the gate of 70.

8.4 Clash Detection Performance

Random 50, 60 or 120-minute events were placed on a 15-minute grid over 30 days (fixed random seed 20261113). The sweep-line detector and the brute-force pairwise check returned identical clash sets at every size.

Events n	Clash pairs	Sweep line (ms)	Pairwise (ms)	Same result
500	444	0.14	4.53	Yes
1,000	1,667	0.31	25.78	Yes
2,000	6,697	0.91	74.34	Yes
4,000	26,822	2.47	299.06	Yes

Table 38: Sweep-line versus brute-force clash detection (one run per size)

8.5 Database Benchmark (Recorded Earlier)

During the hardening stage on 28 October 2026, a benchmark script loaded 120 faculty, 5,000 tasks and two terms. A one-week calendar query took 0.60 ms at the 95th percentile, and claiming 1,000 due notifications took 1.2 ms. This benchmark was not rerun for this report.

8.6 Application Screens

The screens below were captured by an automated Playwright script from the running application, logged in with the seeded test accounts.

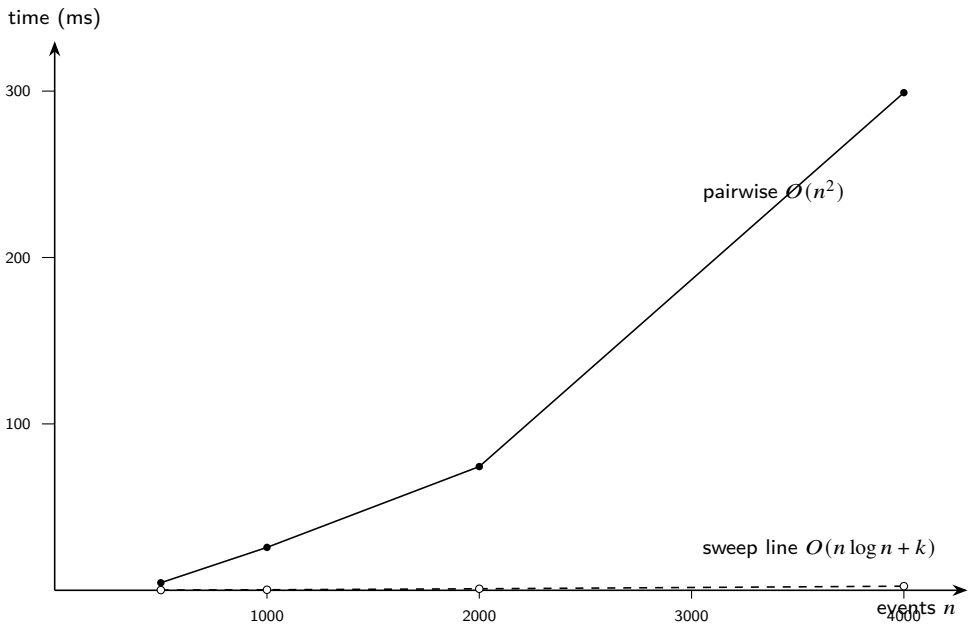


Figure 19: Clash detection time against the number of events

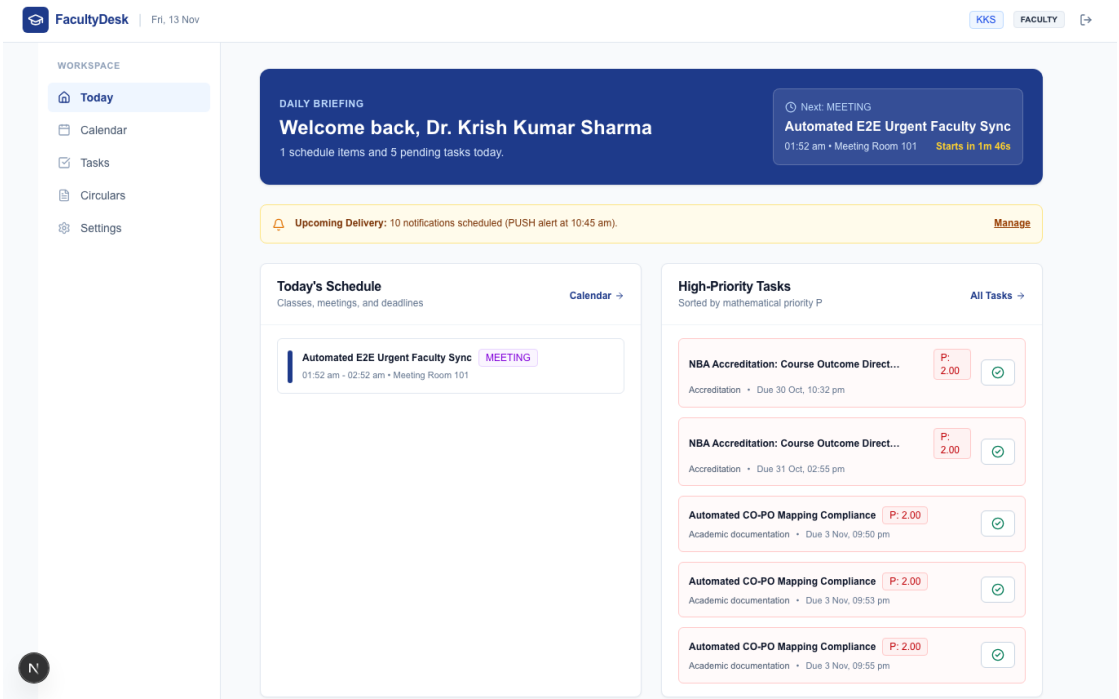


Figure 20: Faculty "Today" screen: next meeting countdown, today's schedule and high-priority tasks

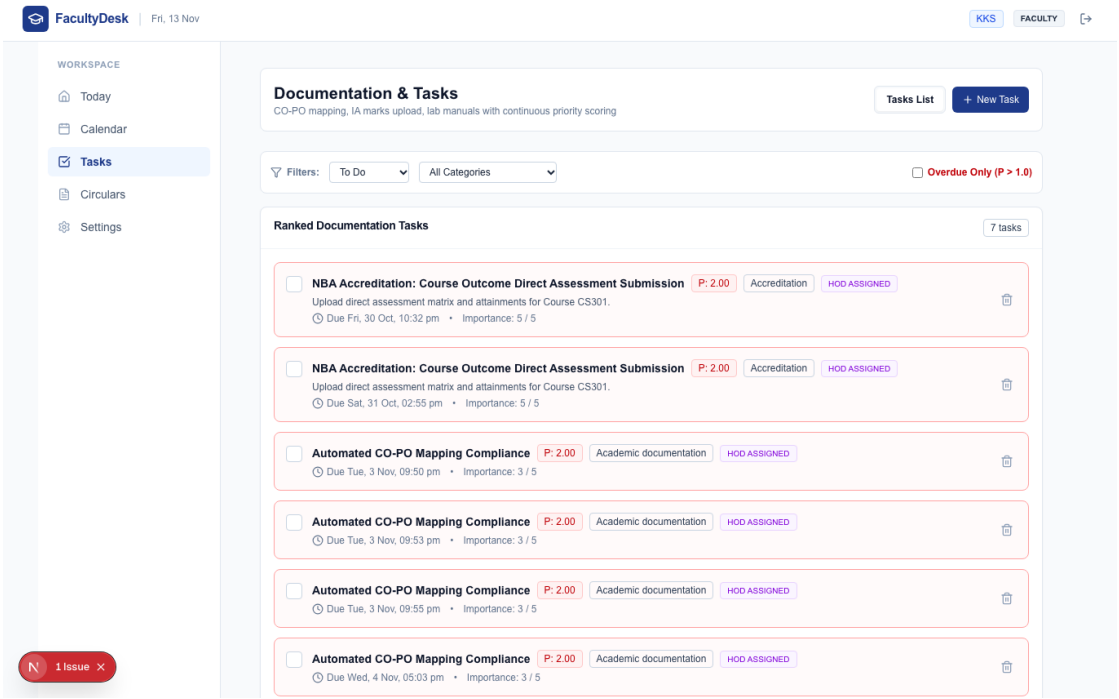


Figure 21: Documentation tasks ranked by priority score P (overdue tasks have $P > 1$)

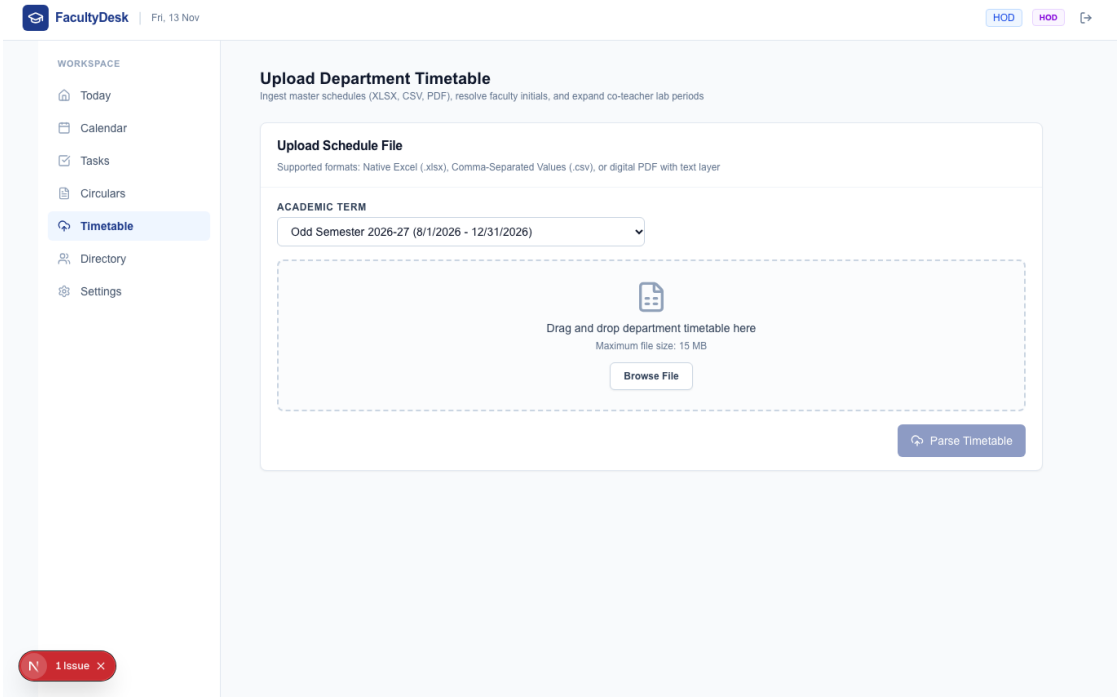


Figure 22: HOD timetable upload screen

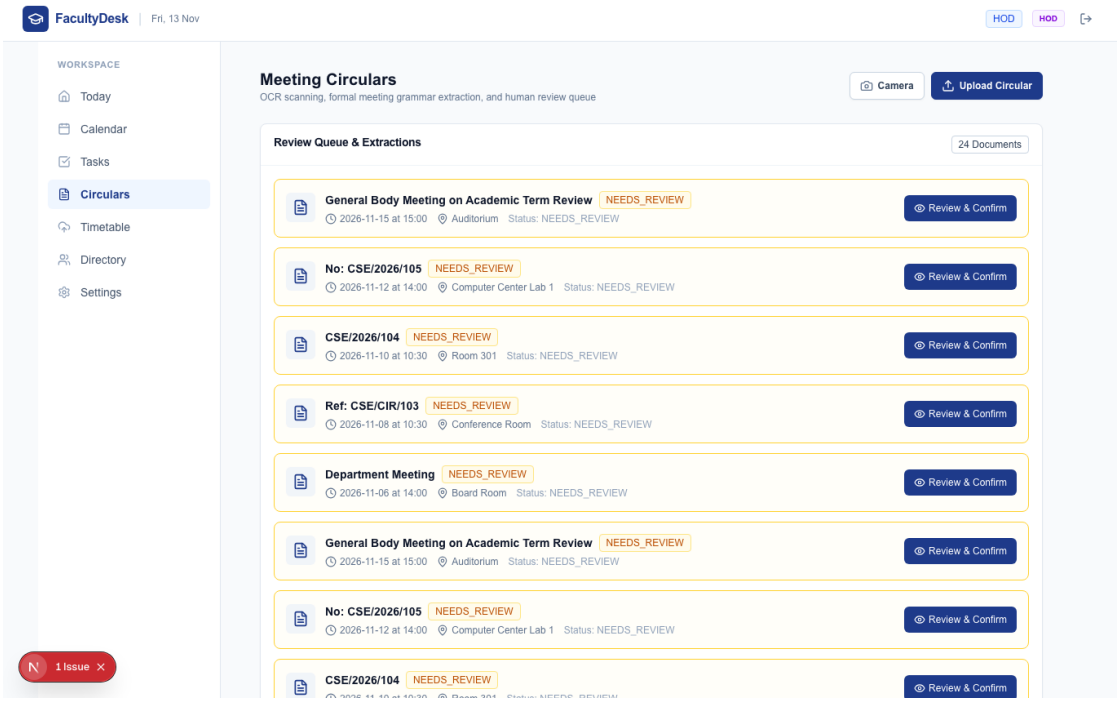


Figure 23: HOD circular review queue with OCR-extracted date, time and venue

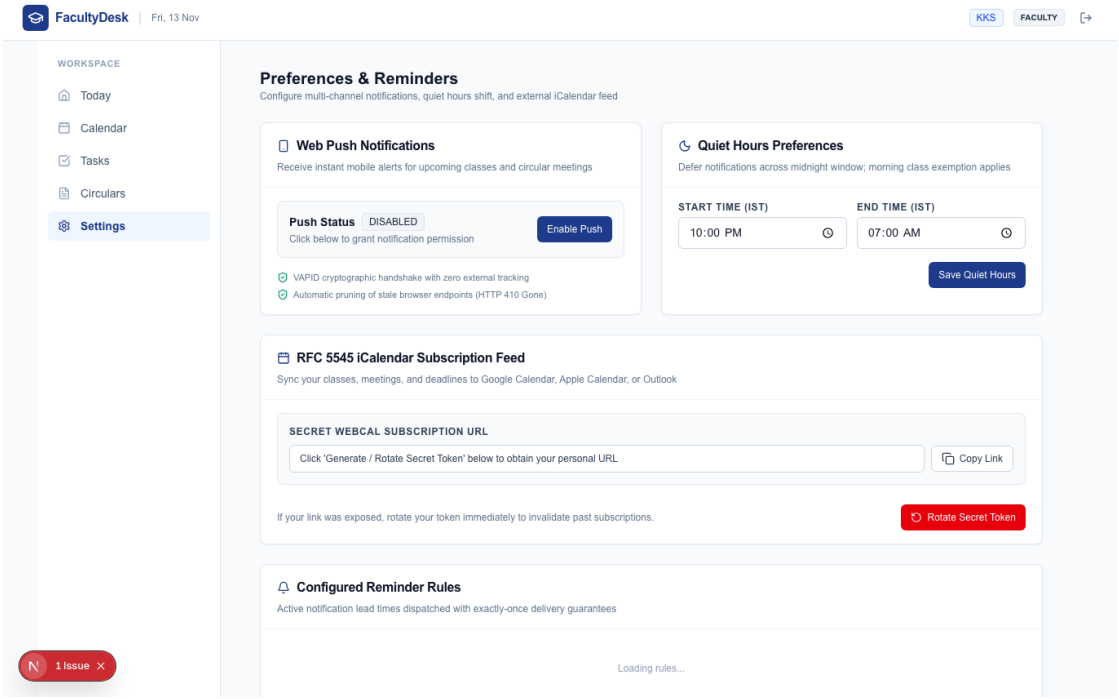


Figure 24: Reminder settings: web push, quiet hours and private ICS feed

9. Discussion of Results

9.1 What the Results Show

The 83 backend tests and the 4 browser journeys confirm that the main business flows work end to end on the running system: a timetable becomes classes on a teacher's calendar, a circular becomes a confirmed meeting, a reminder is recorded as sent, and an assigned task can be completed with its alerts resolved. The object-oriented design is what made this testable: because the domain classes contain no framework code, 68 unit tests check the business rules in 3.3 seconds without a database.

9.2 Clash Detection

The sweep-line detector agreed with the brute-force check on every random instance, and its advantage grew with size: 32 times faster at 500 events and 121 times faster at 4,000 events. This matches the expected $O(n \log n + k)$ against $O(n^2)$ behaviour. One faculty member has only a few hundred events per term, so both methods are fast enough for a single person; the gain matters when the clash check and free-slot finder run across a whole department. The timings come from one run per size on one laptop and should be read as indicative.

9.3 Initials Matching

The worked examples show a deliberate trade-off. Two three-letter initials that differ by one letter have a similarity of 0.667, below the threshold of 0.85, so fuzzy matching helps only with longer tokens such as names or aliases with a typo. For short initials the system relies on exact and alias matches and reports anything else to the HOD. A missed match can be fixed by adding an alias; a wrong match would put classes on the wrong teacher's calendar, so the design prefers the first.

9.4 Limitations Found During Testing

Table 39: Limitations and planned fixes

Limitation	Observation	Planned fix
Reminder retry path	When a send fails, the dispatcher calls `retry()` while the reminder is still SENDING, which the state table does not allow. The failed reminder and the rest of its batch wait for stale recovery (5 minutes) instead of the designed back-off. Reminders are still sent at most once.	Move the reminder to FAILED before `retry()`; add a test with a notifier that fails.
Re-confirming a timetable	Confirming the same timetable again creates duplicate class events, because earlier generated events are not replaced.	On confirmation, replace the previous upload's future class events, keyed by slot and date.
Layering	Four application files import infrastructure classes directly.	Move OCR, parser and storage behind domain interfaces.
No field pilot	OCR accuracy on real department circulars, time saved and user satisfaction were not measured.	Run a one-semester pilot in one department.
Scanned timetables	Timetable PDFs without a text layer are rejected rather than read with OCR.	Add OCR for scanned timetable PDFs.

9.5 Comparison with the 2026 Literature

The timetabling papers surveyed in Section 12 focus on *generating* timetables by optimisation or rules. FacultyDesk takes the timetable as given and focuses on *distributing* it to individuals, combining it with meetings and tasks, and reminding people. The document-AI papers report that modern vision–language models can extract fields from scanned documents and that a multi-stage pipeline with preprocessing improves accuracy. FacultyDesk follows the multi-stage approach with classical Tesseract OCR, which runs on a department server without a GPU; replacing the extractor with a small vision–language model is a possible extension at a higher hardware cost.

10. Conclusion

FacultyDesk shows that a department's existing documents, a timetable spreadsheet and photos of paper circulars, can be turned into a personal, clash-aware calendar with reminders for every faculty member, without anyone retyping information. The system covers eight modules: identity and access, directory and academic calendar, timetable ingestion, circular scanning, calendar, reminders, tasks with escalation, and operations.

Object-oriented analysis and design shaped the whole project. Noun analysis of the problem statement produced twenty domain classes that hold their own rules, three of which enforce their life cycles as state machines. The UML models (use case, class, sequence, collaboration, statechart, activity, package, component and deployment diagrams) describe the system from the user's view down to the deployed containers, and every name in them matches the code. A four-layer architecture kept the business rules free of web, database and queue code, and the Repository, Unit of Work, Strategy, Factory, Composite and State patterns each solved a concrete problem: storage independence, atomic multi-object updates, new file formats and channels without changing existing code, and safe status changes.

On 13 November 2026 all 83 backend tests and all 4 end-to-end journeys passed on the running system, and the sweep-line clash detector matched brute force exactly while running up to 121 times faster on 4,000 events.

10.1 Future Work

- Fix the reminder retry path and the duplicate classes on re-confirmation, with regression tests.
- Run a pilot in one department to measure OCR accuracy on real circulars, reminder delivery and time saved for the HOD.
- Read scanned timetable PDFs with OCR, and evaluate a small vision–language model for circular extraction.
- Complete the layering by moving OCR, parsing and storage behind domain interfaces.
- Add SMS or WhatsApp as a further notification channel through a new `Notifier` strategy.

11. References

[1] Grady Booch, James Rumbaugh, Ivar Jacobson, *The Unified Modeling Language User Guide*, 2nd Edition, Addison-Wesley, 2005.

[2] Craig Larman, *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*, 3rd Edition, Prentice Hall, 2004.

[3] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

[4] Martin Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.

[5] Robert C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017.

[6] Ray Smith, "An Overview of the Tesseract OCR Engine," *Proc. 9th International Conference on Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.

[7] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[8] B. Desruisseaux (Ed.), "Internet Calendaring and Scheduling Core Object Specification (iCalendar)," IETF RFC 5545, 2009.

[9] J. Shen, P. Yuan, A. Ghosh, Y. Mai, D. Dahlmeier, "OCR or Not? Rethinking Document Information Extraction in the MLLMs Era with Real-World Large-Scale Datasets," arXiv:2603.02789, March 2026.

[10] Y. Han, Y. Zhang, Y. Wang, Y. Jin, K. Z. Ke, J. Zhao, "A Multistage Extraction Pipeline for Long Scanned Financial Documents: An Empirical Study in Industrial KYC Workflows," arXiv:2604.26462, April 2026.

[11] Y. Fehlis *et al.*, "Operationalizing Document AI: A Microservice Architecture for OCR and LLM Pipelines in Production," arXiv:2605.18818, May 2026.

[12] M. Elhüseyni, "Course Timetabling under Conflict and Lecturer Preferences using Bi-objective Optimization Model," arXiv:2607.03901, July 2026.

[13] H. Sharafuddin, "Automated University Timetabling: An AI Approach for Efficient Schedule Management," *International Journal of Emerging Technologies in Learning (iJET)*, vol. 21, no. 2, pp. 91–99, 2026, doi: 10.3991/ijet.v21i02.59329.

12. Literature Survey Proofs

Five papers published in 2026 that relate to FacultyDesk were surveyed: two on university timetabling and three on extracting information from scanned documents. Each entry below was checked on the paper's official page (the journal page or the arXiv page) on 13 November 2026. The title, authors, date and identifier are copied from that page, and a screenshot of the page is included as proof. The summaries are our own, written from the abstracts.

12.1 Automated University Timetabling: An AI Approach for Efficient Schedule Management

Authors	Hassan Sharafuddin (Arab Open University, Kuwait)
Published	29 April 2026, *International Journal of Emerging Technologies in Learning (iJET)*, vol. 21, no. 2, pp. 91–99
Identifier	DOI: 10.3991/ijet.v21i02.59329
Summary	Describes Arab Open University's move from manual timetabling to a rule-based Python scheduling system serving about 62,000 students and over 1,000 staff across nine branches. The author reports timetable creation falling from four to six weeks to under two hours, 85% fewer conflicts, and room utilisation rising from 65% to 78%, and argues that transparent rules suit institutions better than black-box optimisers.
Relevance to FacultyDesk	Supports our choice of transparent, explainable rules (initials tiers, review gate, priority formula). The paper builds timetables; FacultyDesk distributes them to each teacher.

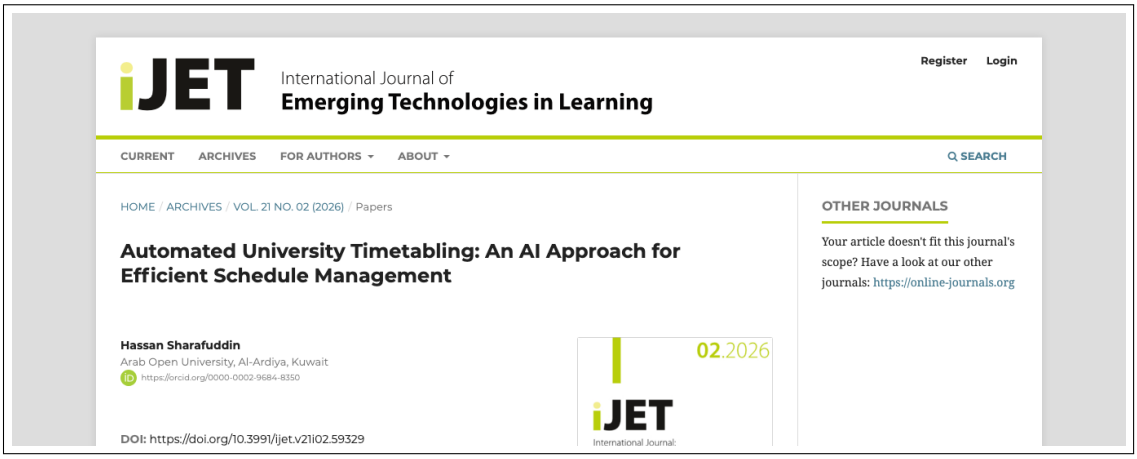


Figure 25: Proof: official page of "Automated University Timetabling: An AI Approach for Efficient Schedule Management"

12.2 Course Timetabling under Conflict and Lecturer Preferences using Bi-objective Optimization Model

Authors	Murat Elhüseyni
Published	Submitted 4 July 2026, arXiv (Optimization and Control)
Identifier	arXiv: 2607.03901
Summary	Formulates a real course timetabling problem with lecturer rest periods and theory-before-practical precedence as a bi-objective mixed-integer linear program. On a real dataset it reduces course conflicts by 13.33% and improves lecturer schedule compactness by 26% compared with the actual timetable.
Relevance to FacultyDesk	Its notion of a conflict matches our half-open interval test for clash detection. It generates timetables, whereas FacultyDesk reads and distributes them.

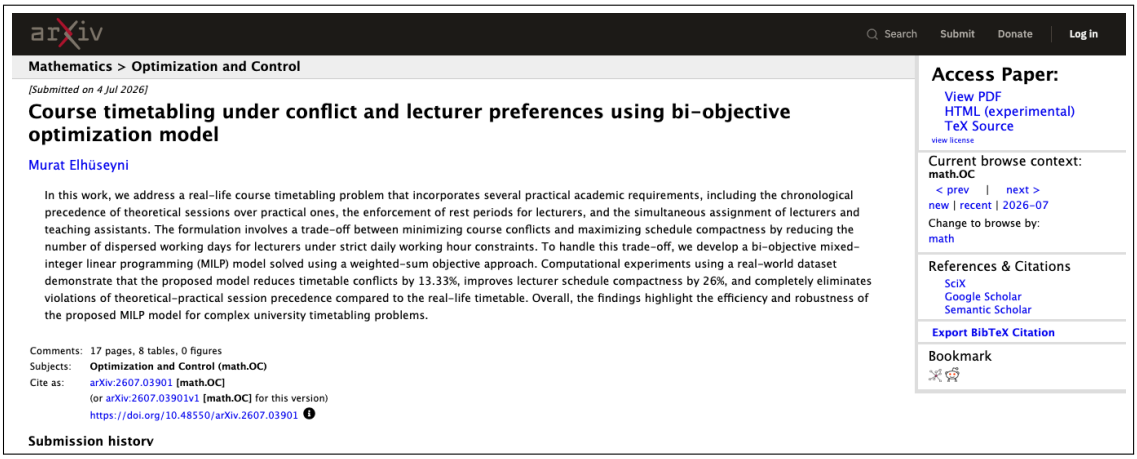


Figure 26: Proof: official page of “Course Timetabling under Conflict and Lecturer Preferences using Bi-objective Optimization Model”

12.3 OCR or Not? Rethinking Document Information Extraction in the MLLMs Era with Real-World Large-Scale Datasets

Authors	Jiyuan Shen, Peiyue Yuan, Atin Ghosh, Yifan Mai, Daniel Dahlmeier
Published	Submitted 3 March 2026, arXiv (Computation and Language)
Identifier	arXiv: 2603.02789
Summary	Benchmarks multimodal large language models on business-document information extraction and finds that image-only input can perform comparably to pipelines that add OCR text, with careful schemas and instructions improving results.
Relevance to FacultyDesk	A possible future replacement for our Tesseract-plus-rules extractor, at the cost of GPU hardware.

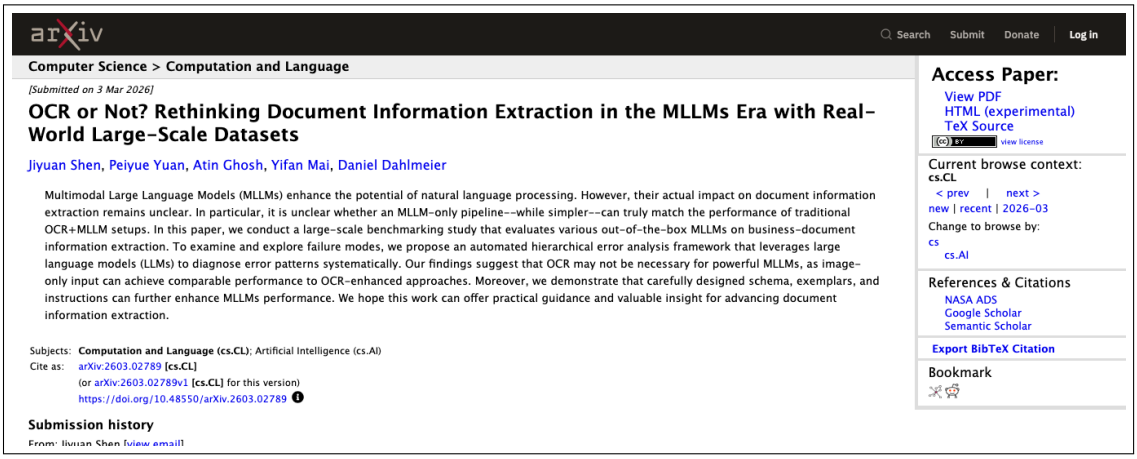


Figure 27: Proof: official page of "OCR or Not? Rethinking Document Information Extraction in the MLLMs Era with Real-World Large-Scale Datasets"

12.4 A Multistage Extraction Pipeline for Long Scanned Financial Documents: An Empirical Study in Industrial KYC Workflows

Authors	Yuxuan Han, Yuanxing Zhang, Yushuo Wang, Yichao Jin, Kenneth Zhu Ke, Jingyuan Zhao
Published	Submitted 29 April 2026, arXiv (Computer Vision)
Identifier	arXiv: 2604.26462
Summary	Proposes a multistage pipeline (image preprocessing, multilingual OCR, page retrieval and compact vision–language extraction) for long scanned KYC documents. On 120 production documents (about 3,000 pages) it beats direct PDF-to-model baselines by up to 31.9 percentage points; the best setup reaches 87.27% field accuracy.
Relevance to FacultyDesk	Supports our design of preprocessing images before OCR and separating text extraction from field extraction.

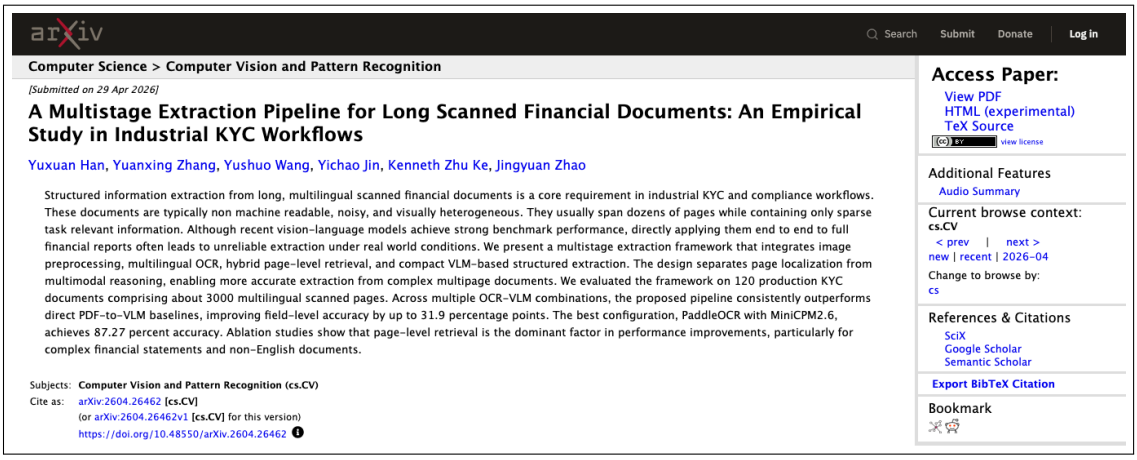


Figure 28: Proof: official page of "A Multistage Extraction Pipeline for Long Scanned Financial Documents: An Empirical Study in Industrial KYC Workflows"

12.5 Operationalizing Document AI: A Microservice Architecture for OCR and LLM Pipelines in Production

Authors	Yao Fehlis, Benjamin Bengfort, Zhangzhang Si, Vahid Eyorokon, Prema Roman, Patrick Deziel, Devon Slonaker, Steve Veldman, Ben Johnson, Joyce Rigelo, Michael Wharton, Steve Kramer
Published	Submitted 12 May 2026, arXiv (Artificial Intelligence)

Identifier	arXiv: 2605.18818
Summary	Presents a microservice architecture for production document AI with classification, OCR and language-model field extraction, separating CPU orchestration from GPU inference and using asynchronous I/O. Profiling shows that OCR, not language-model parsing, dominates end-to-end latency.
Relevance to FacultyDesk	Matches our decision to run OCR asynchronously in Celery workers so that web requests return immediately with `202 Accepted`.

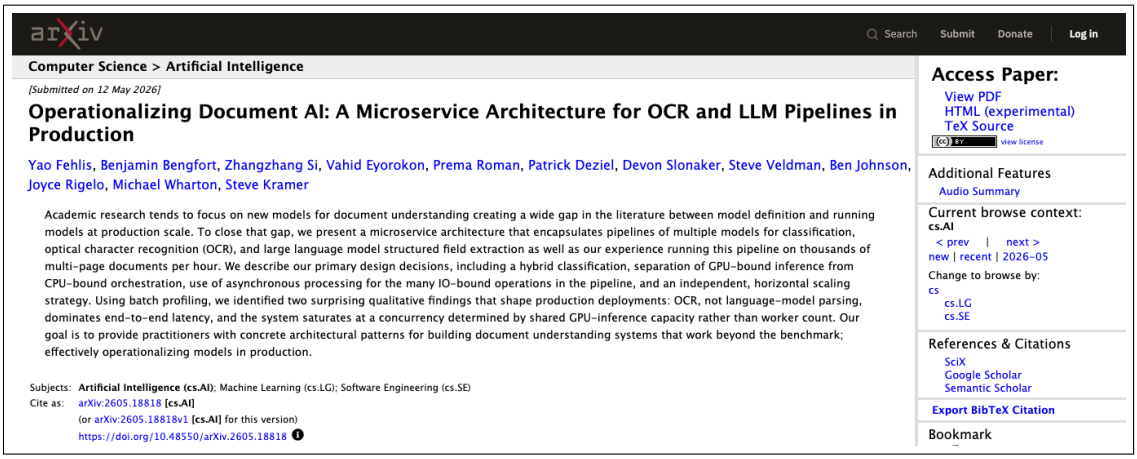


Figure 29: Proof: official page of "Operationalizing Document AI: A Microservice Architecture for OCR and LLM Pipelines in Production"

Gap Addressed by FacultyDesk

The timetabling work optimises how a timetable is built, and the document-AI work improves how fields are read from scans. None of the surveyed papers addresses what happens after both: delivering one combined, clash-checked schedule to each faculty member with reliable reminders and documentation tracking. That is the part FacultyDesk covers.